Efficient Memory Management for Large Language Model Serving with PagedAttention

Presented by: Raghav Tirumale, Kiran Hombal, Vasu Dar

TABLE OF CONTENTS

01

Intro to KV Cache

Quick recap.

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

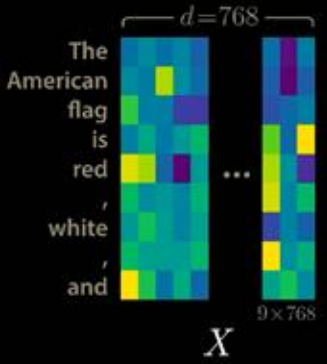
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

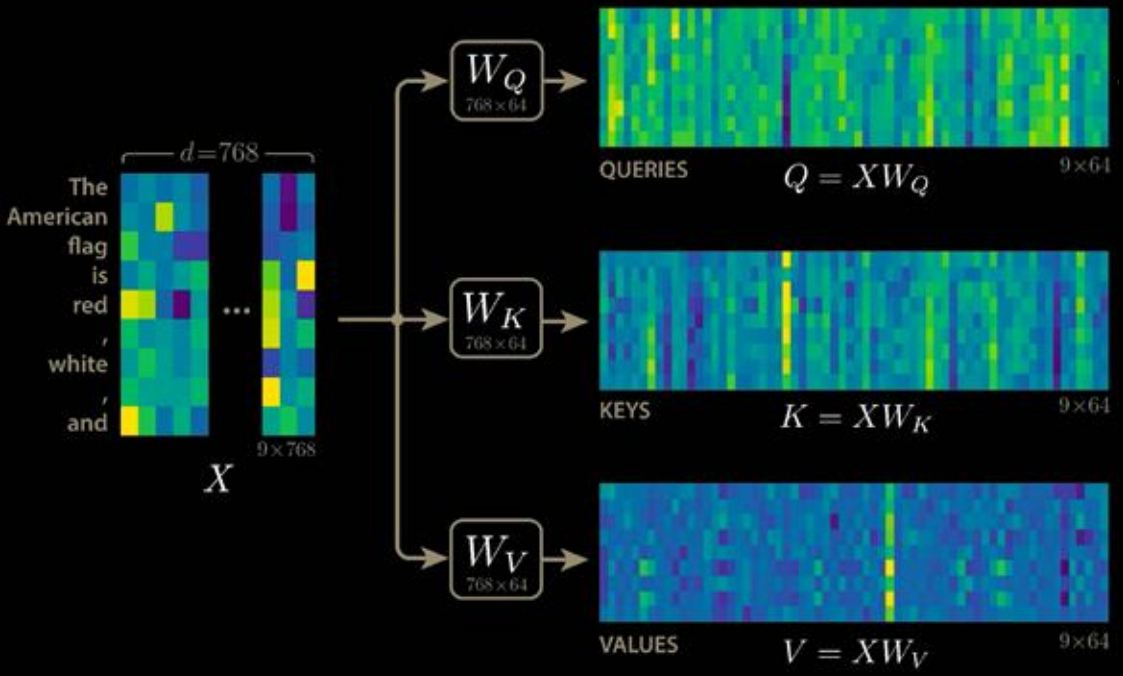
Source: BRUCE ALMIGHTY (2003)

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



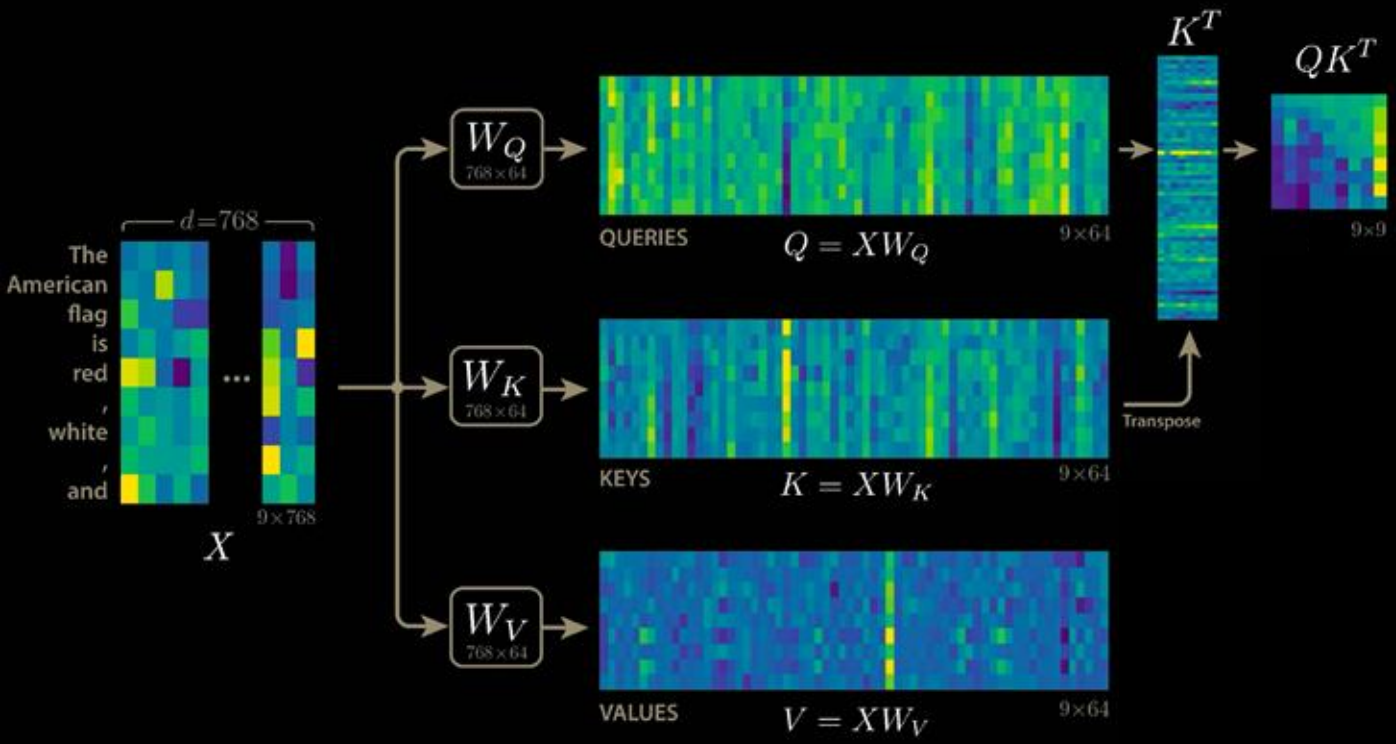
Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



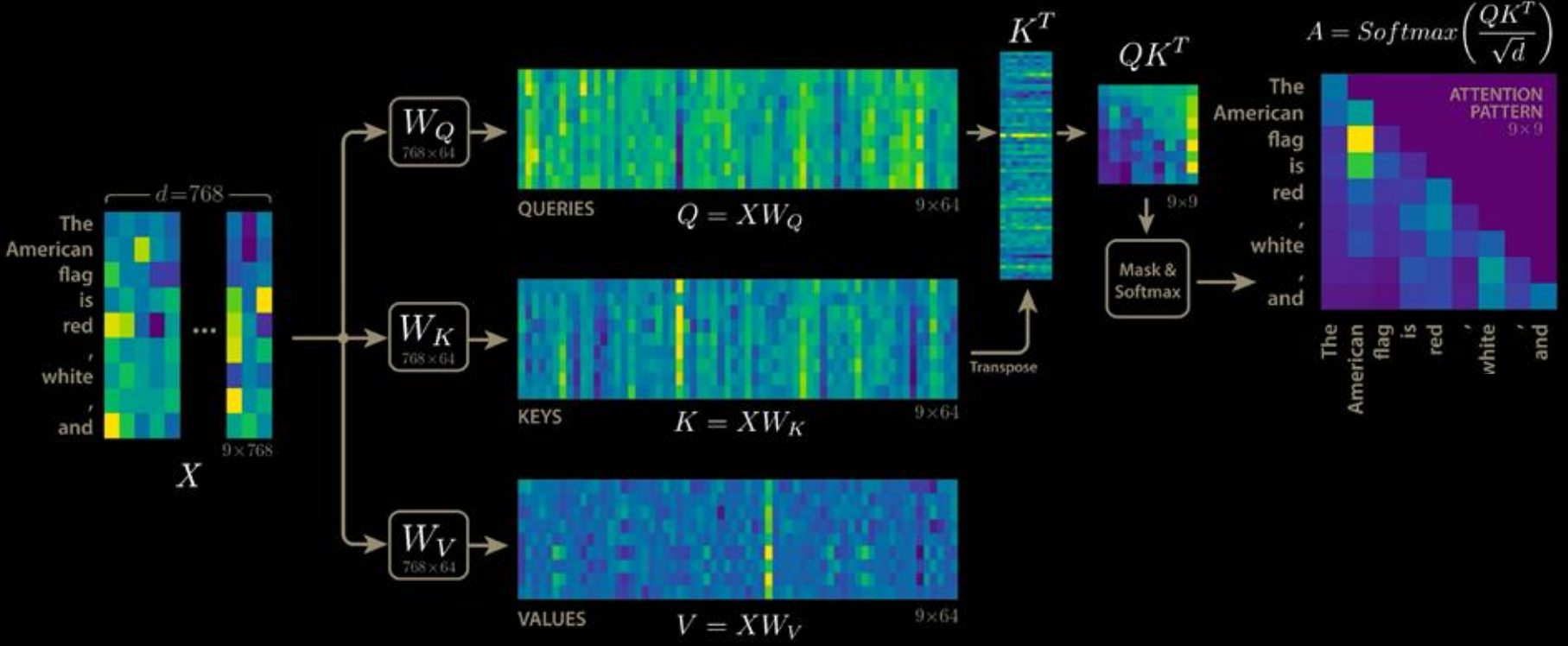
Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



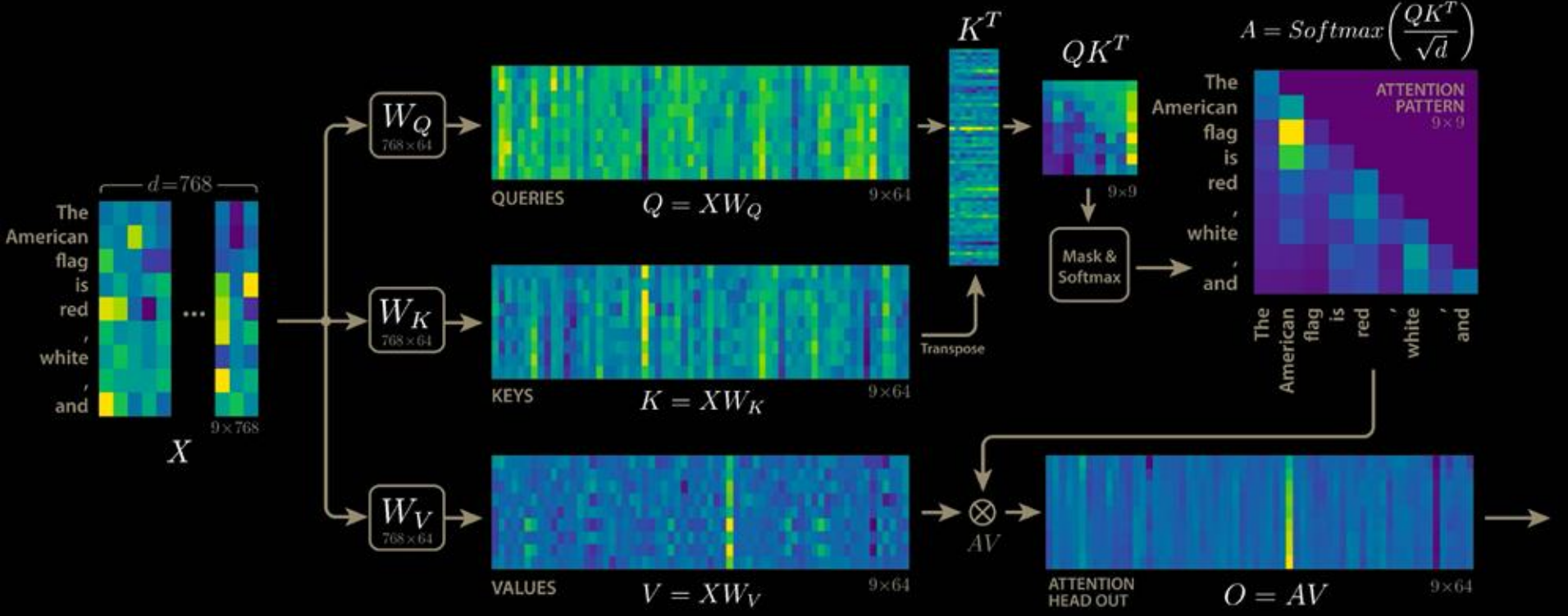
Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

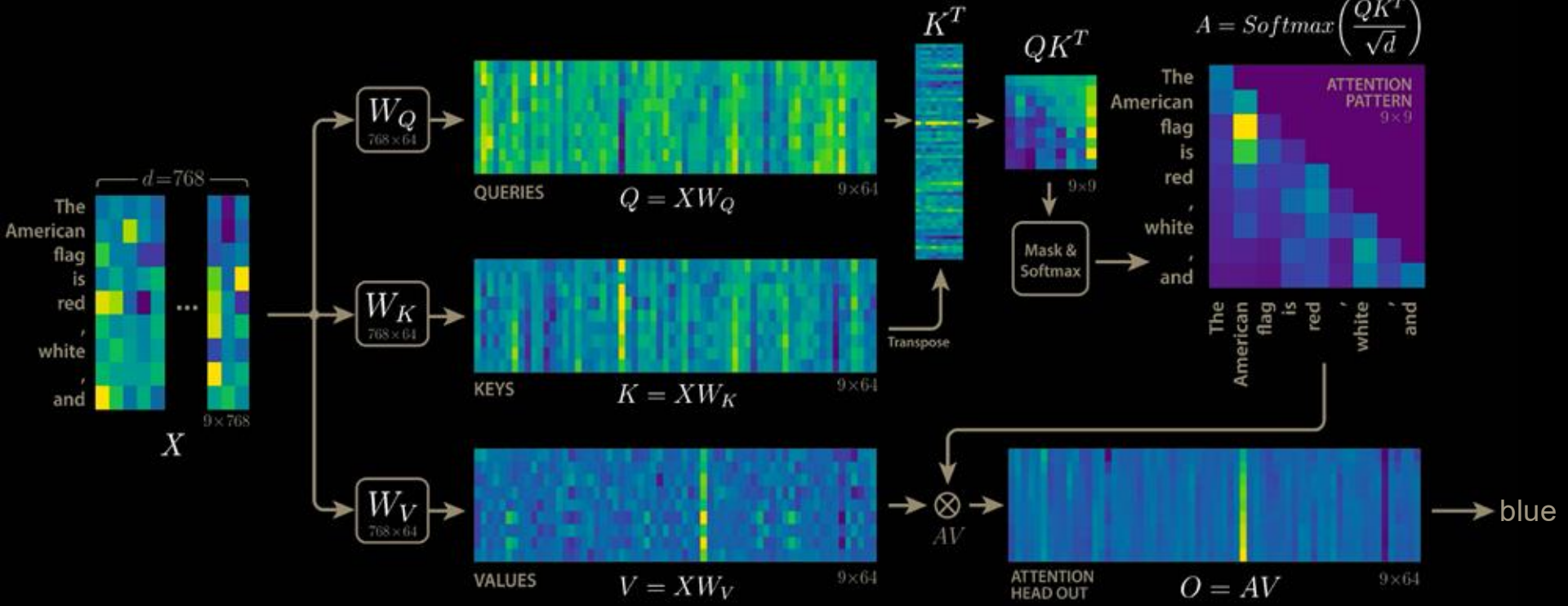


Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]

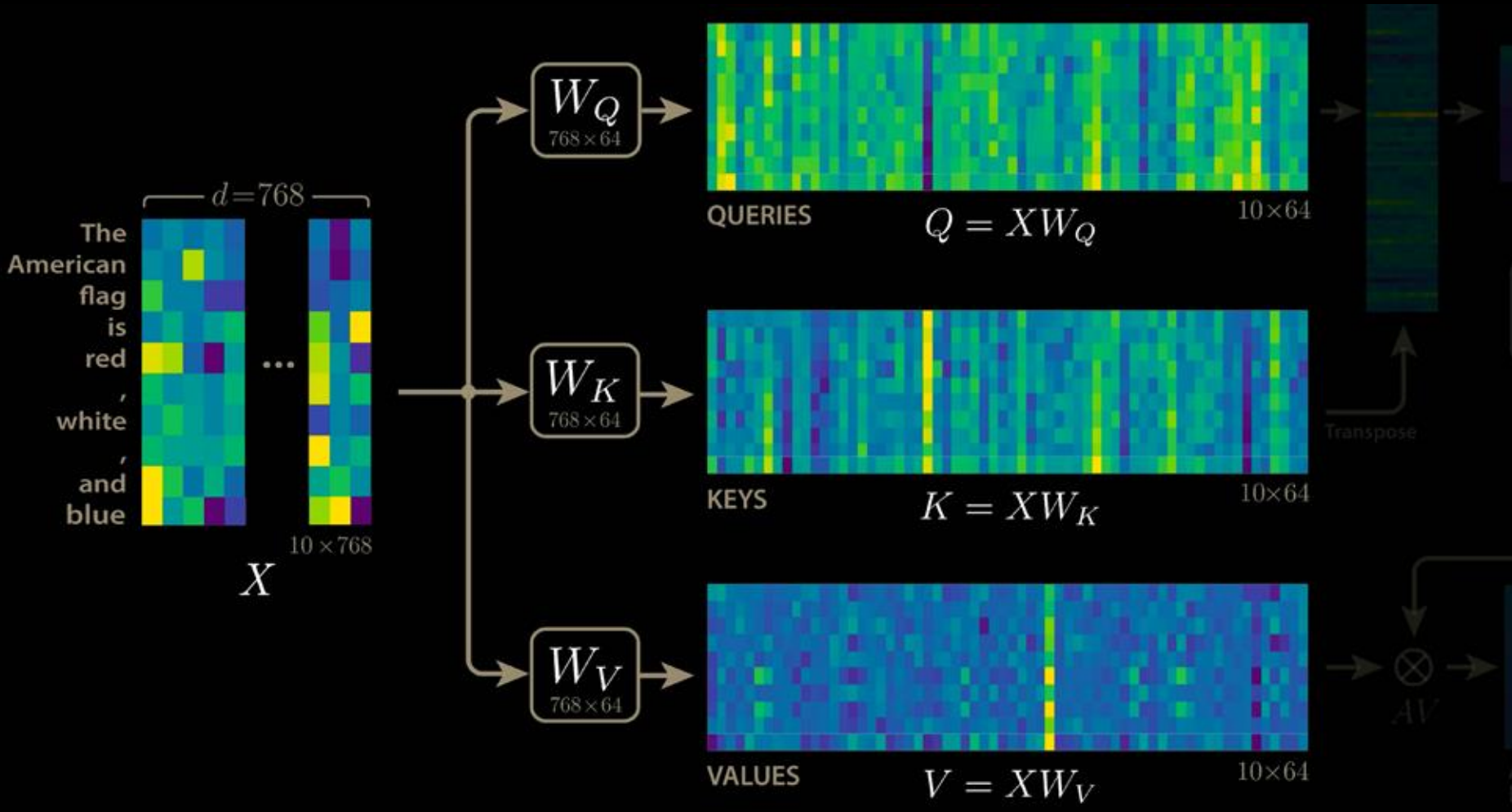
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



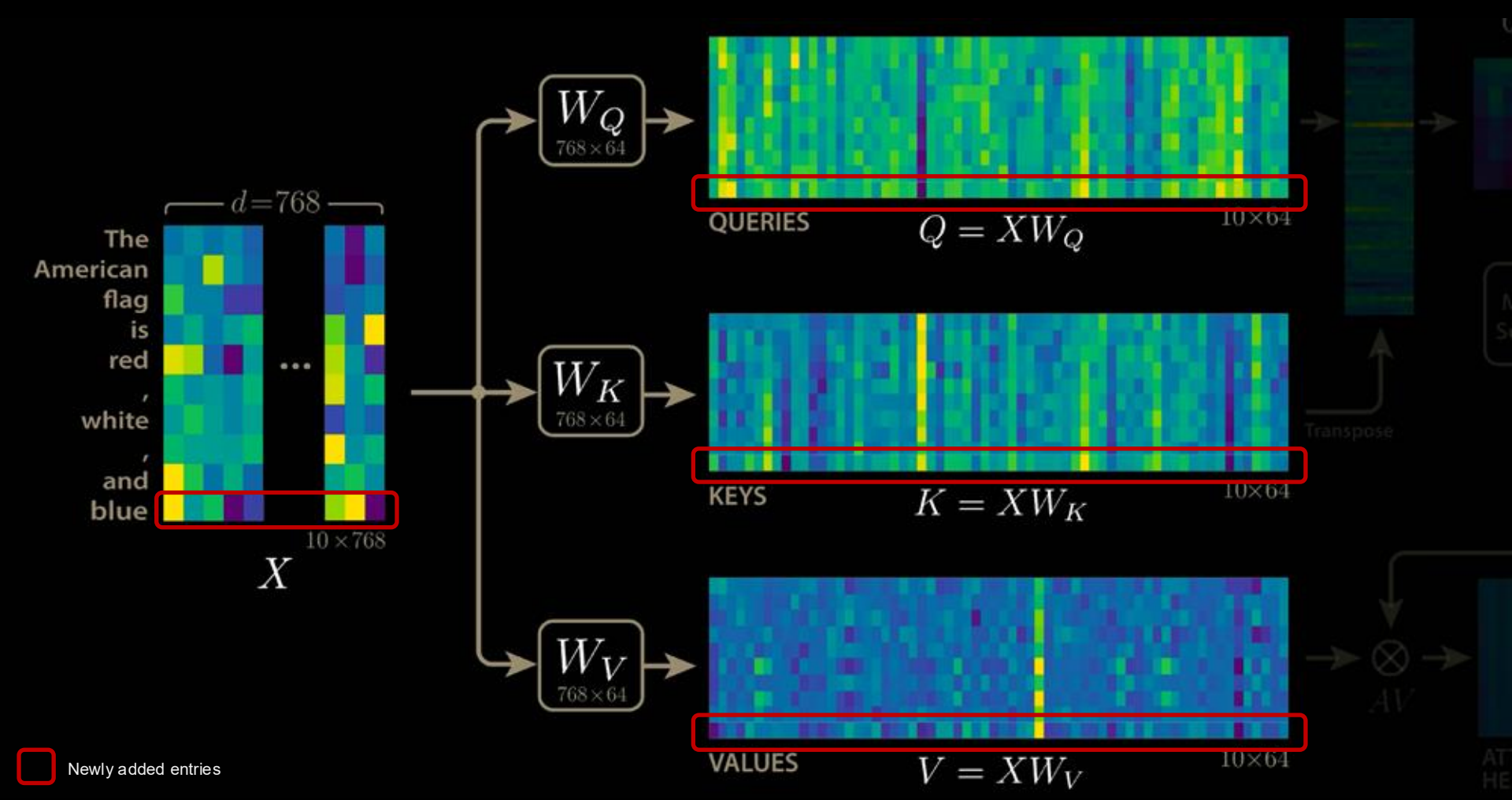
Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]



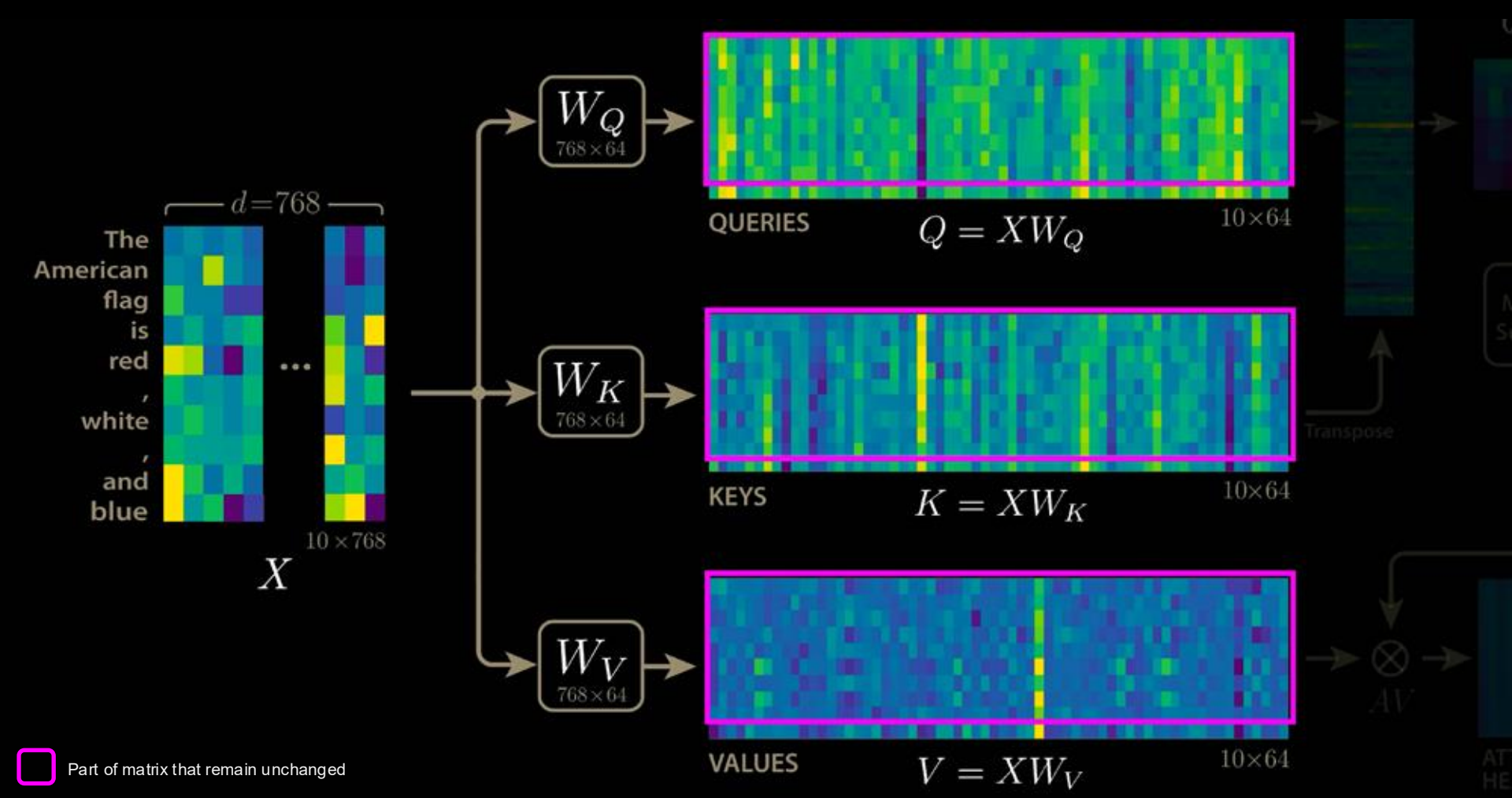
Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]



Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]

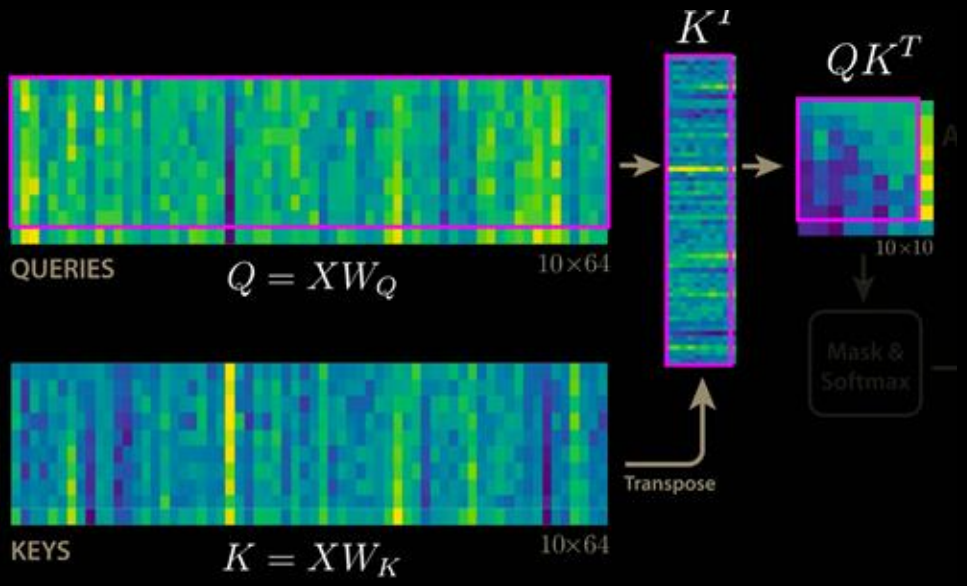


Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]



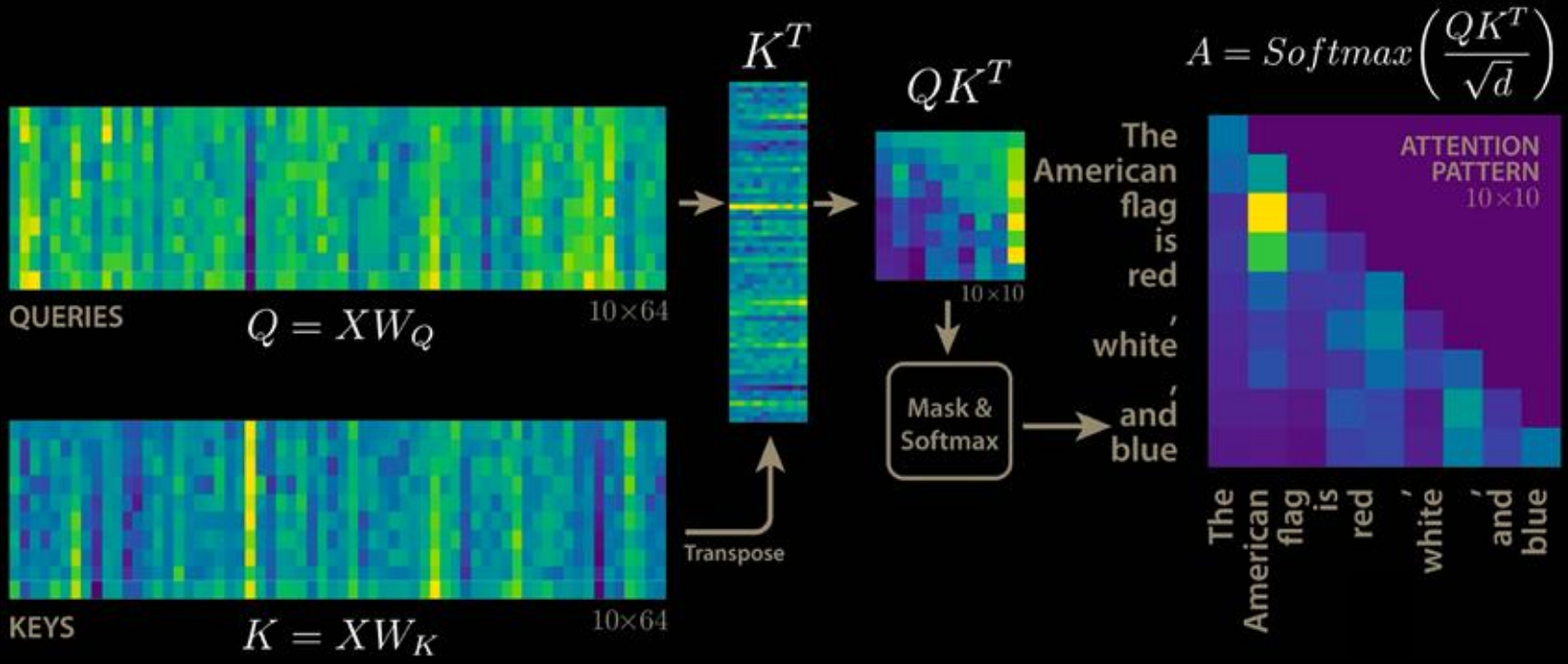
Part of matrix that remain unchanged

Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]

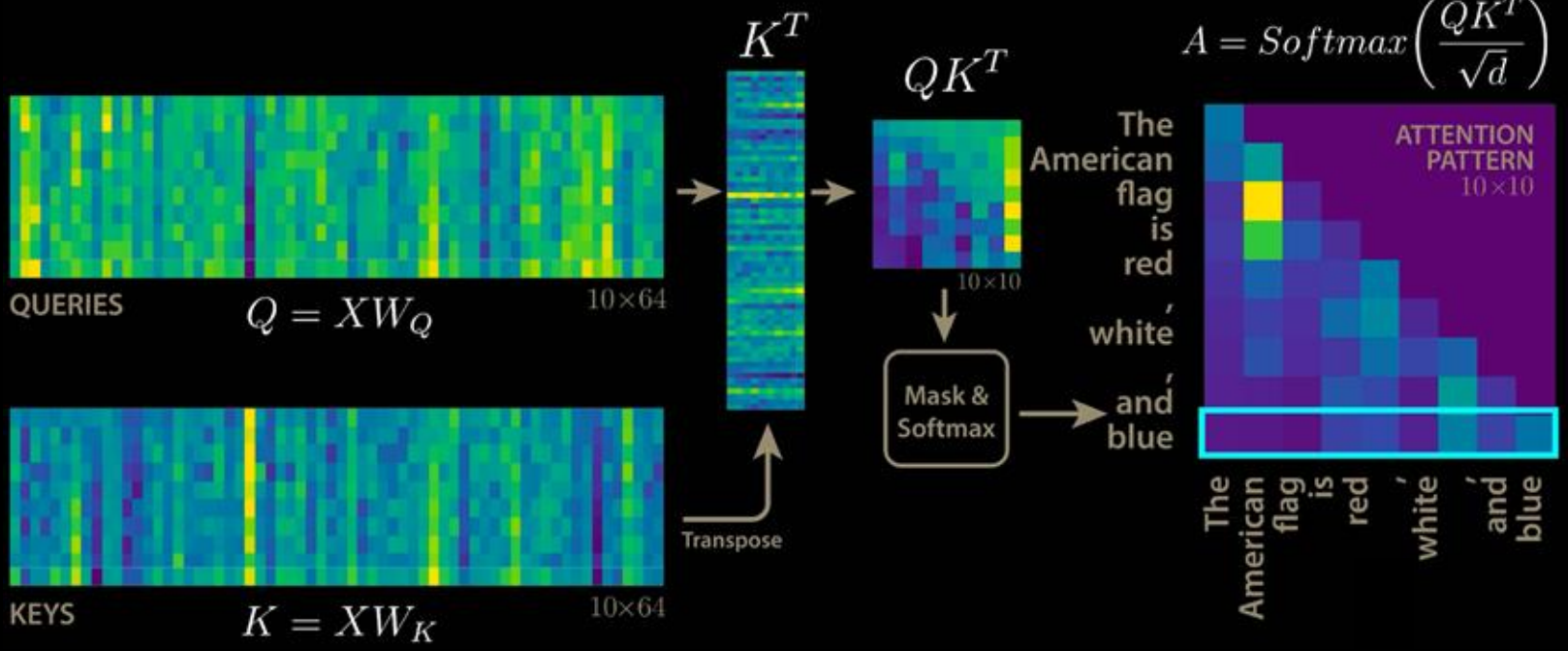


Part of matrix that remain unchanged

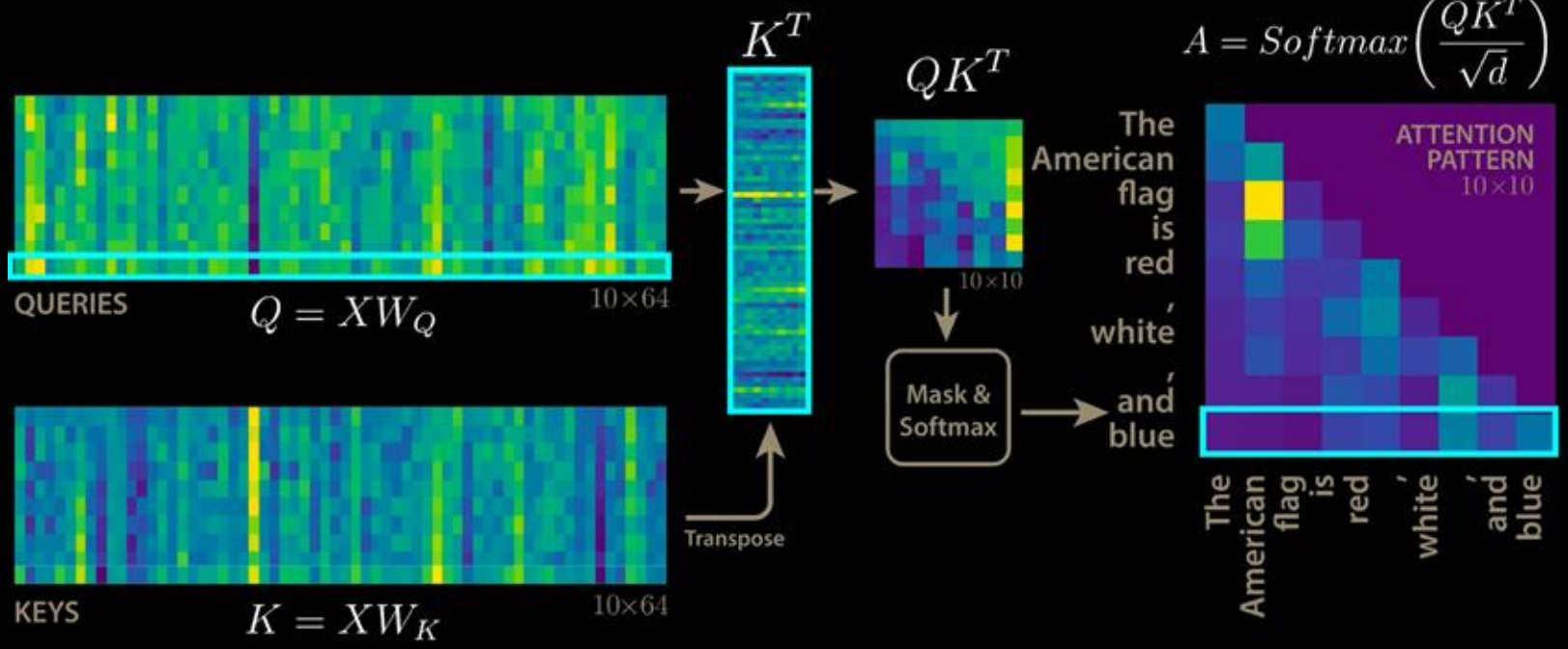
Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]



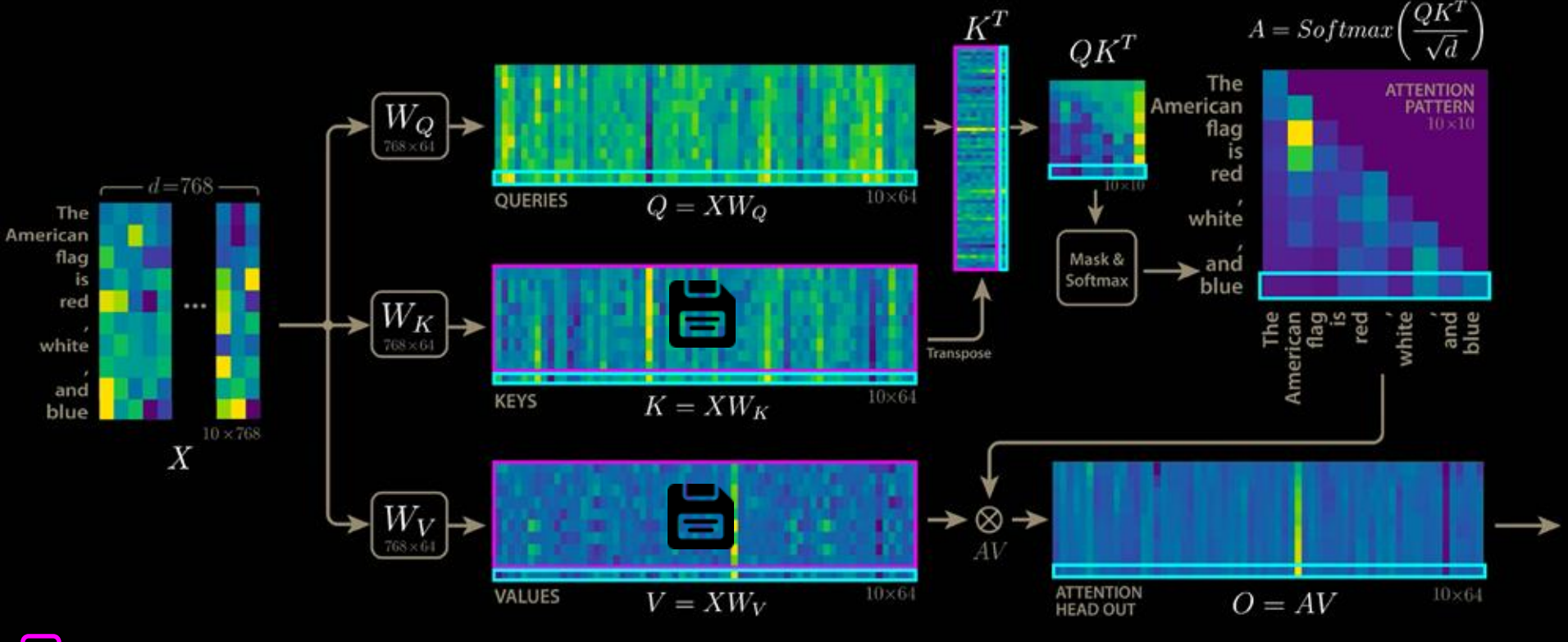
Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]



Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]



Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]

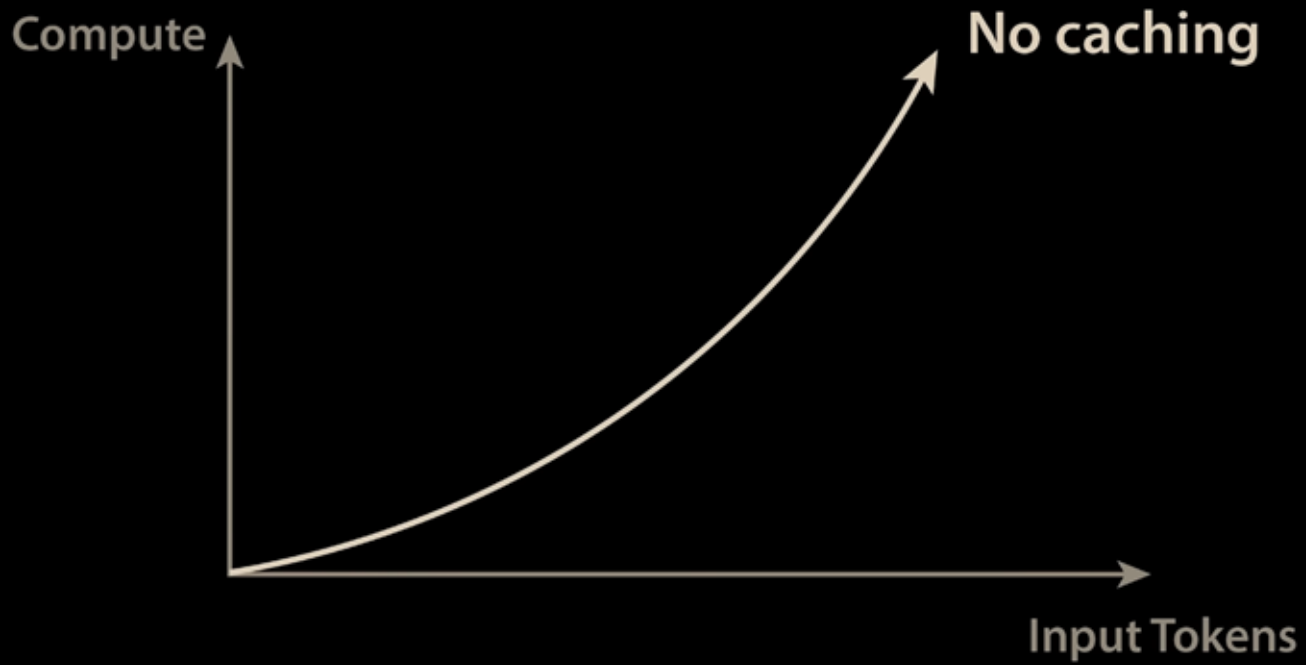


Cached in memory

New computations

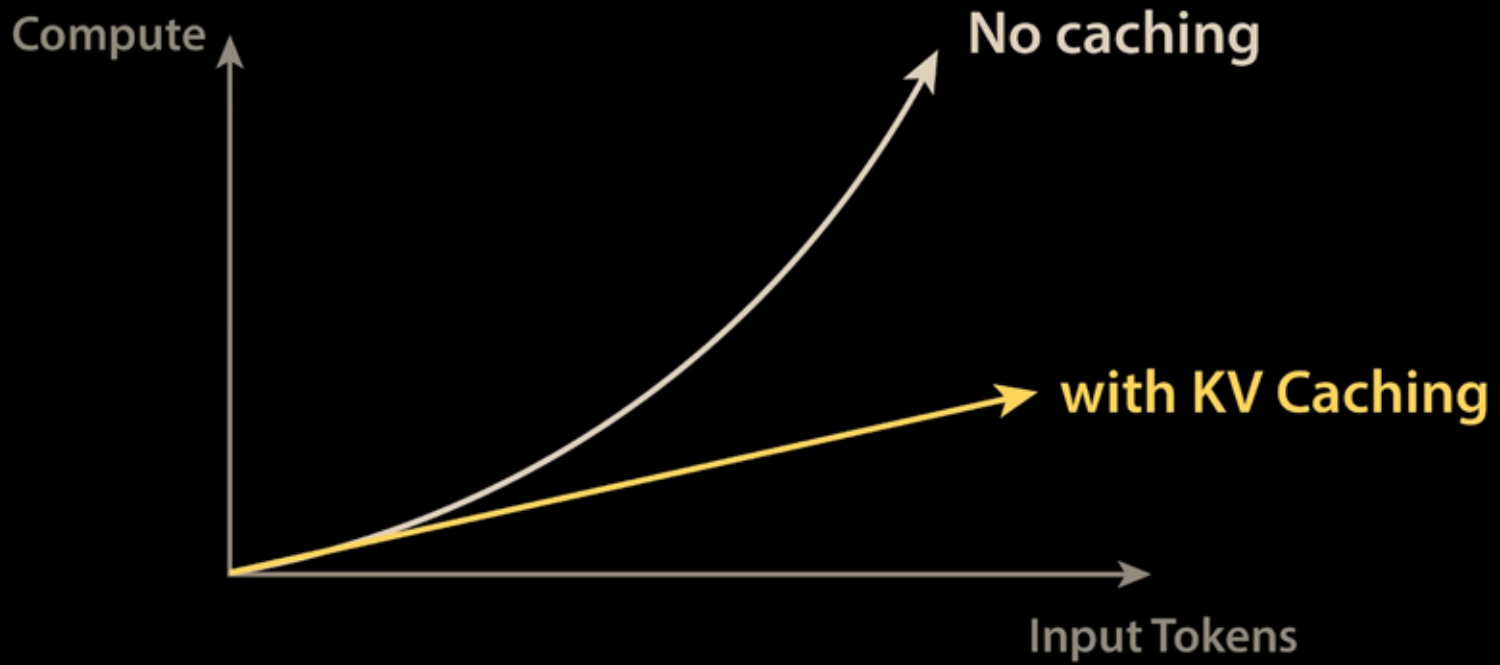
Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]

ATTENTION BLOCK COMPUTE SCALING

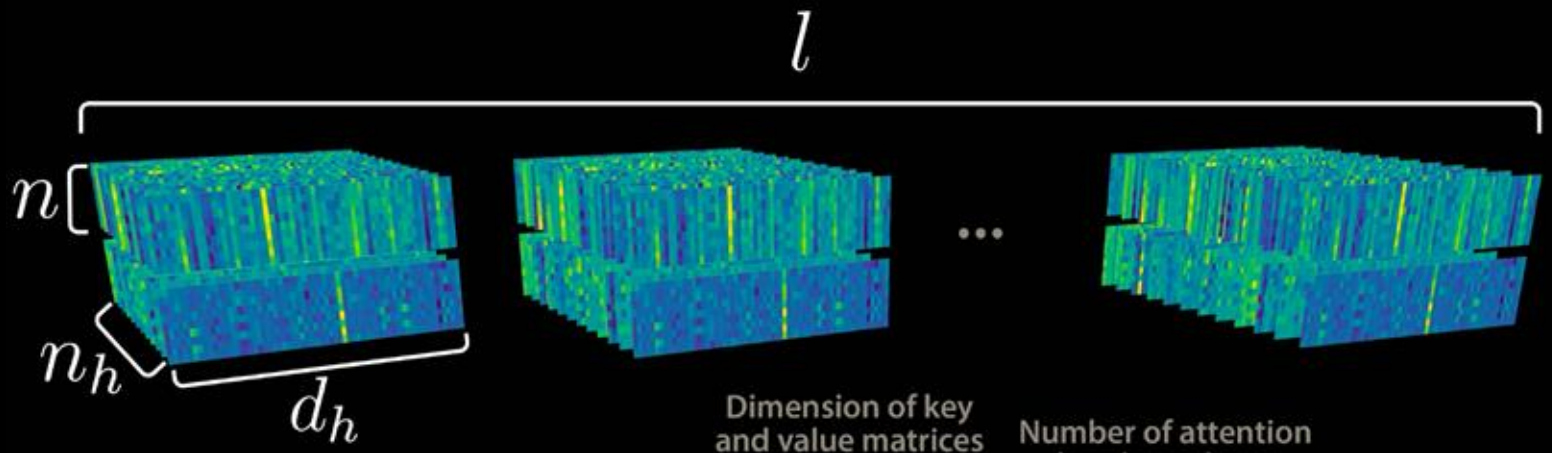


Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]

ATTENTION BLOCK COMPUTE SCALING



Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]



Input tokens

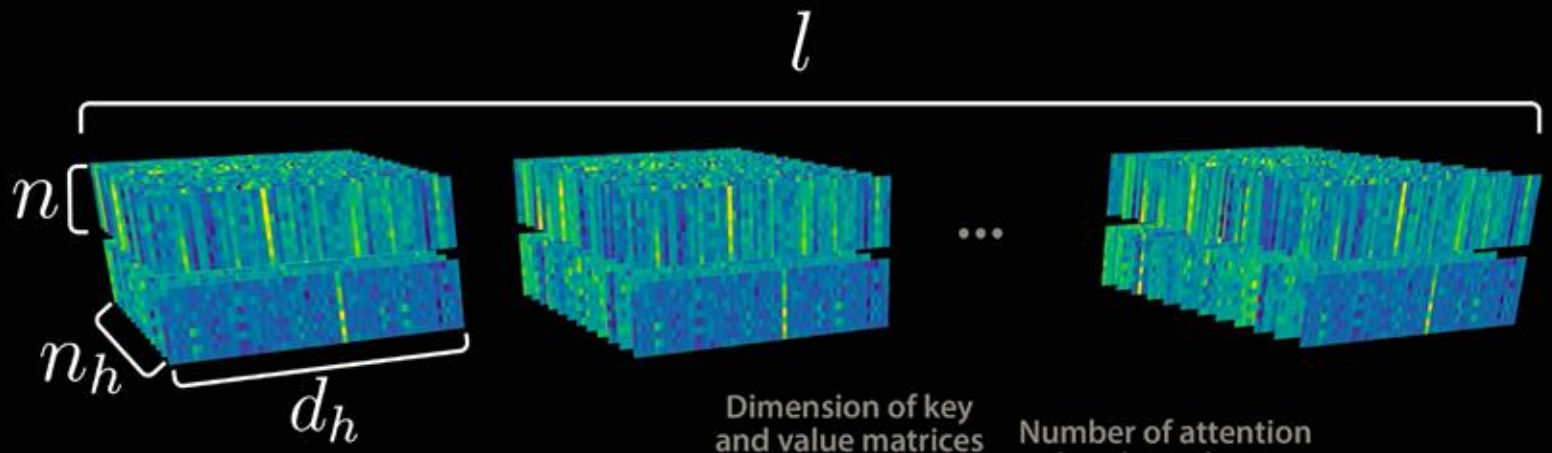
Dimension of key and value matrices

Number of attention heads per layer

$$\text{NUMBER OF ENTRIES IN CACHE} = 2 \cdot n \cdot d_h \cdot n_h \cdot l$$

← Number of layers

Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]



Input tokens

Dimension of key and value matrices

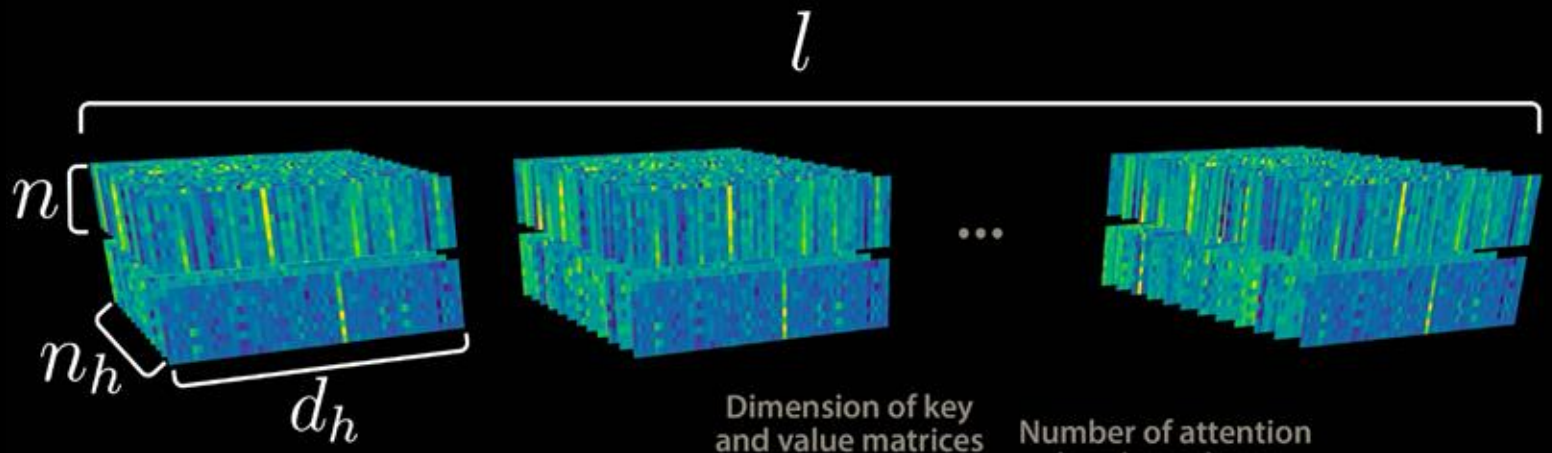
Number of attention heads per layer

$$\text{NUMBER OF ENTRIES IN CACHE} = 2 \cdot n \cdot d_h \cdot n_h \cdot l$$

← Number of layers

→ $let\ \ d_h = 128,\ n_h = 128,\ l = 61,\ n = 100000$ (DeepSeek R1/V3 Architecture)

Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]



Input tokens

Dimension of key and value matrices

Number of attention heads per layer

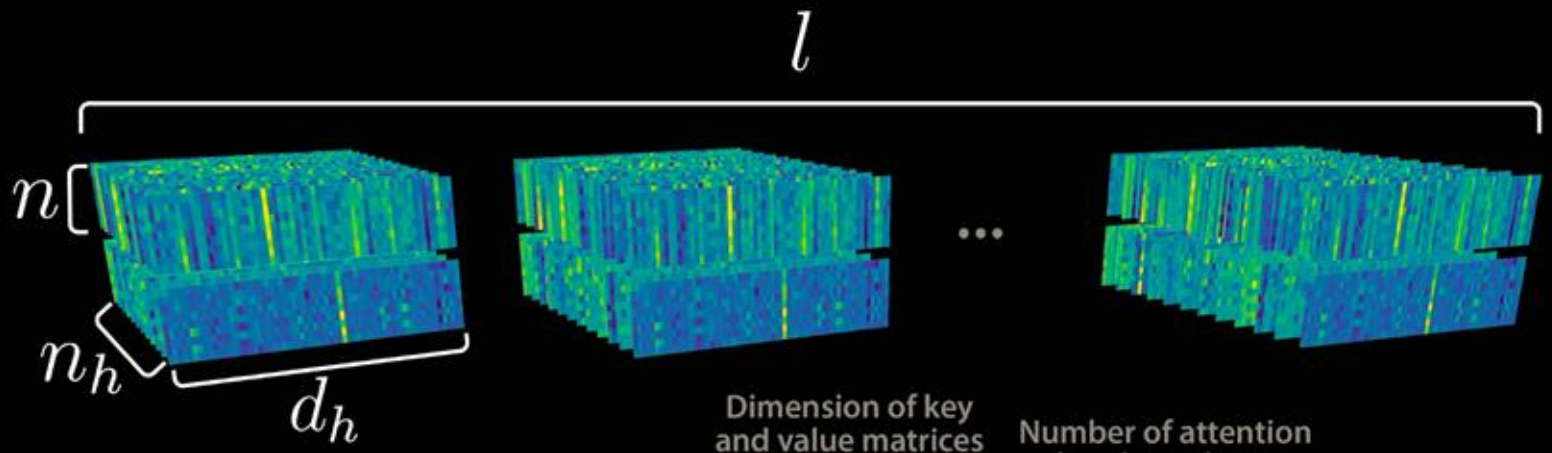
$$\text{NUMBER OF ENTRIES IN CACHE} = 2 \cdot n \cdot d_h \cdot n_h \cdot l$$

← Number of layers

→ $let\ \ d_h = 128,\ n_h = 128,\ l = 61,\ n = 100000$ (DeepSeek R1/V3 Architecture)

CACHE SIZE $= (2)(128)(128)(61)(2) = 3.998 \times 10^6\ Bytes/token \approx 4\ MB/token$

Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]



Input tokens

Dimension of key and value matrices

Number of attention heads per layer

$$\text{NUMBER OF ENTRIES IN CACHE} = 2 \cdot n \cdot d_h \cdot n_h \cdot l$$

← Number of layers

→ $let\ \ d_h = 128,\ n_h = 128,\ l = 61,\ n = 100000$ (DeepSeek R1/V3 Architecture)

CACHE SIZE $= (2)(128)(128)(61)(2) = 3.998 \times 10^6\ Bytes/token \approx 4\ MB/token$

$$4\ MB/token \times 100000\ tokens = 400\ GB$$

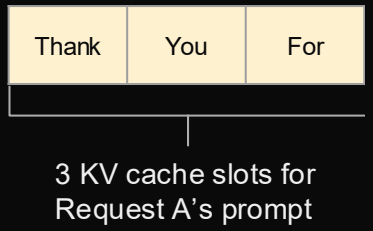
↑ Context length

Source: Welch Labs - How DeepSeek Rewrote the Transformer [MLA]

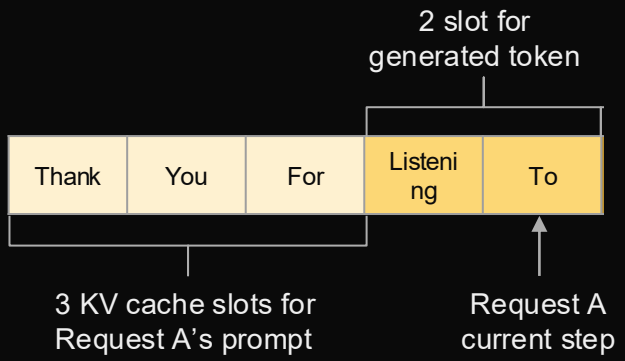
02

How was memory managed before PagedAttention

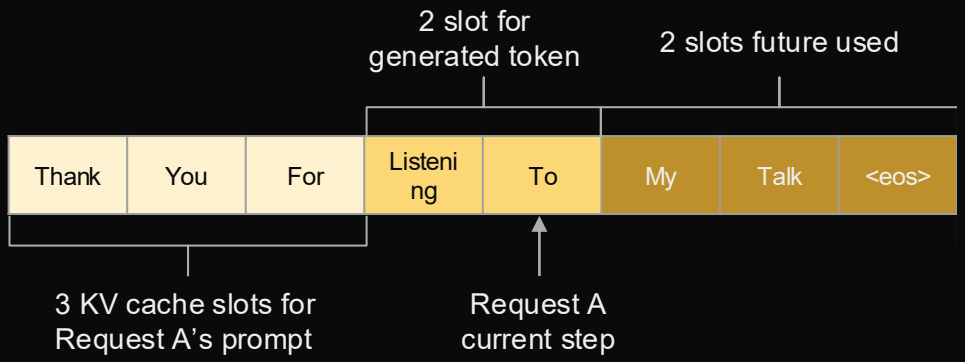
KV Cache management in previous systems



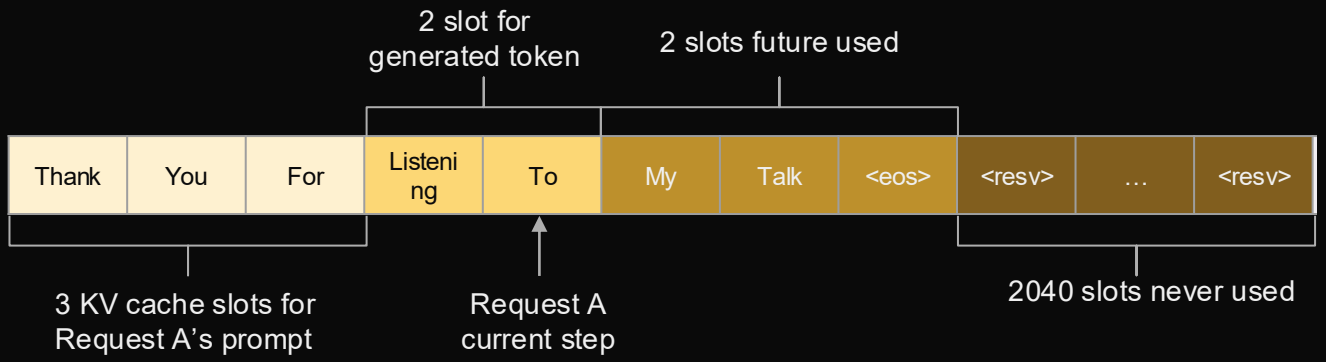
KV Cache management in previous systems



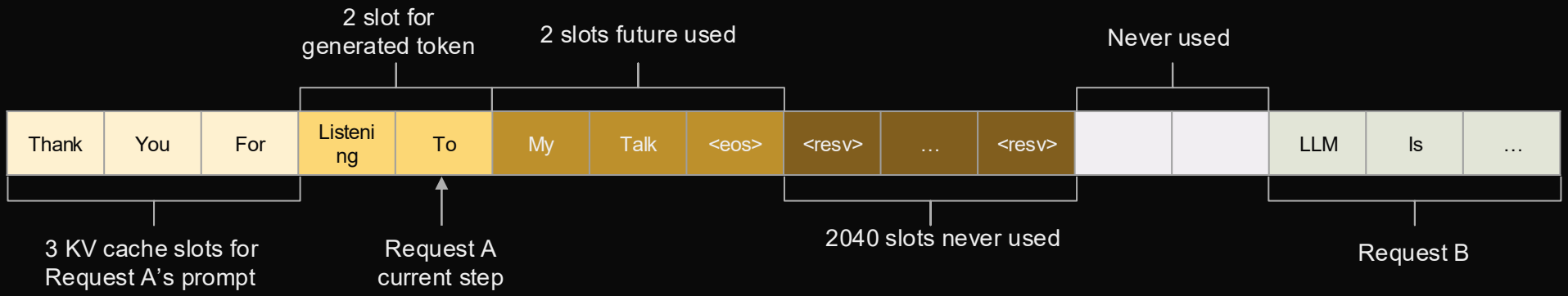
KV Cache management in previous systems



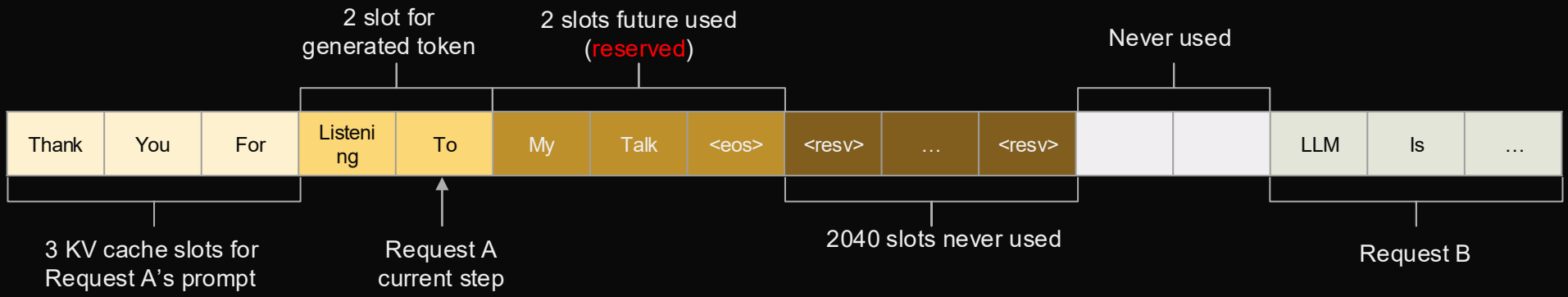
KV Cache management in previous systems



KV Cache management in previous systems

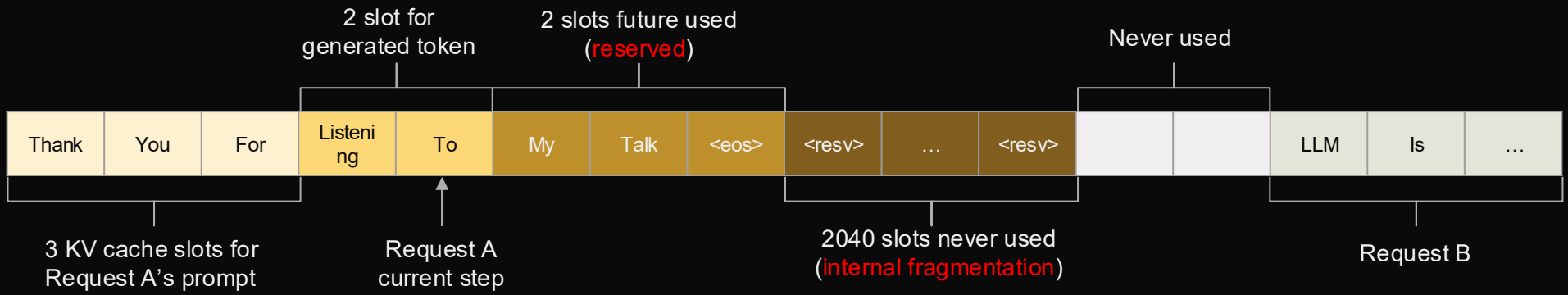


KV Cache management in previous systems



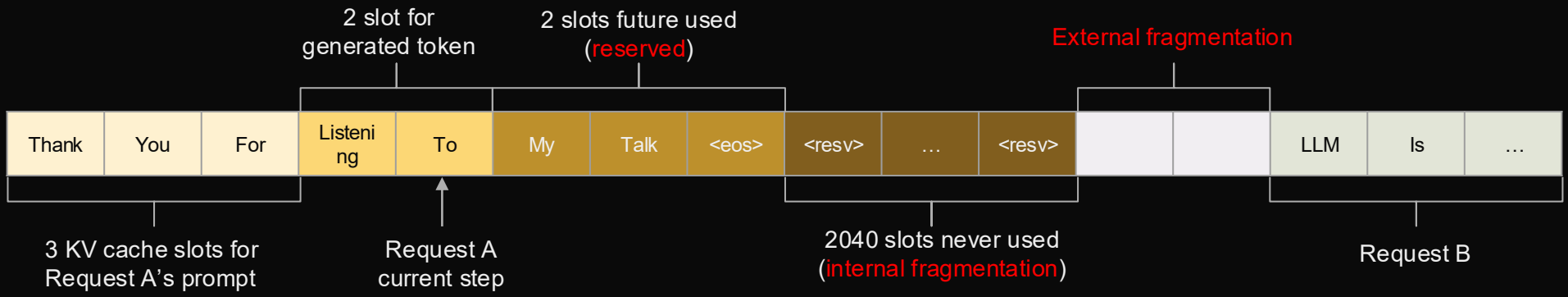
- Reservation: not used at the current step but used in the future.

KV Cache management in previous systems



- Reservation: not used at the current step but used in the future.
- Internal Fragmentation: over-allocated due to the unknown output length.

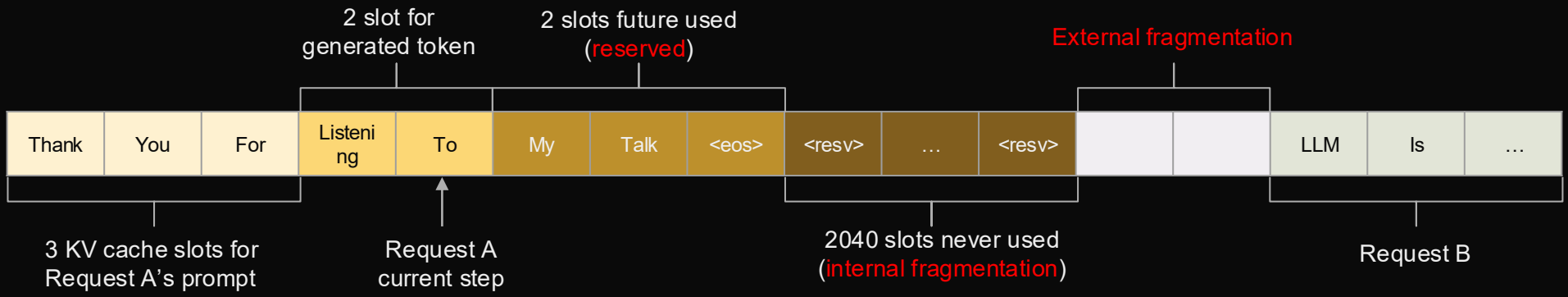
KV Cache management in previous systems



- Reservation: not used at the current step but used in the future.
- Internal Fragmentation: over-allocated due to the unknown output length.
- External fragmentation: due to different sequence lengths at an allocator level.

Memory layout: | Req A | Req B | Req C | FREE 1GB |

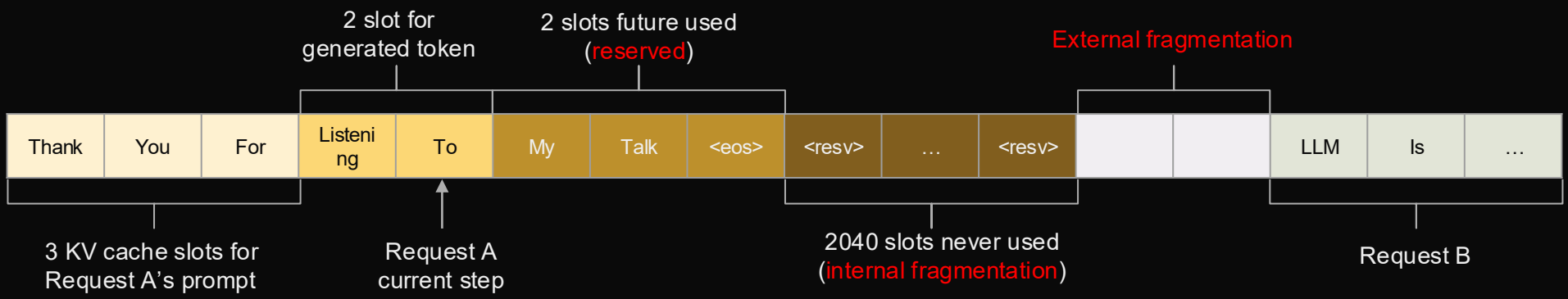
KV Cache management in previous systems



- Reservation: not used at the current step but used in the future.
- Internal Fragmentation: over-allocated due to the unknown output length.
- External fragmentation: due to different sequence lengths at an allocator level.

Memory layout: | Req A | FREE 1GB | Req C | FREE 1GB |

KV Cache management in previous systems

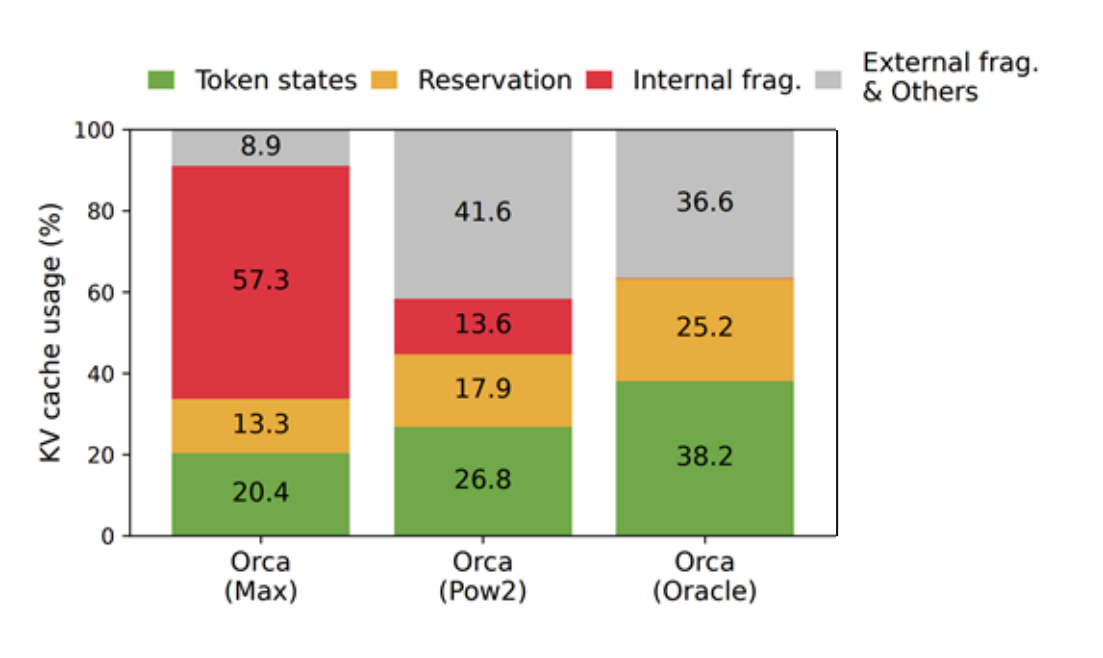


- Reservation: not used at the current step but used in the future.
- Internal Fragmentation: over-allocated due to the unknown output length.
- External fragmentation: due to different sequence lengths at an allocator level.

Memory layout: | Req A | FREE 1GB | Req C | FREE 1GB |

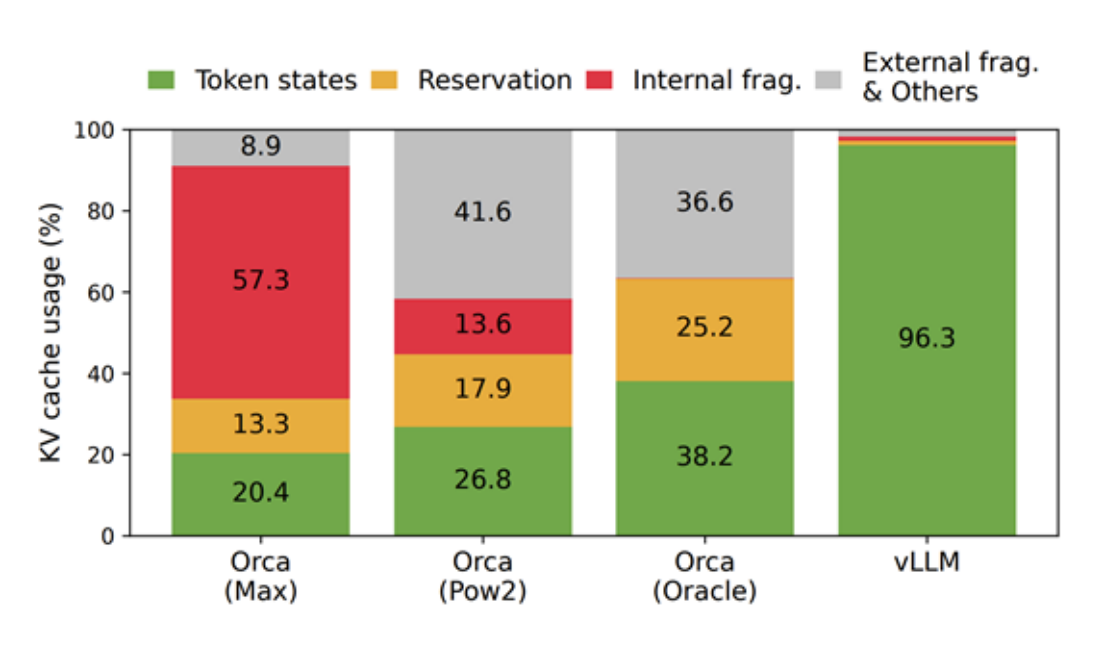
New request(1.1GB) – cannot be allocated, even though you have 2GB available space.

Significant memory wasted in KV Cache space



Only **20% - 40%** of KV cache is utilized to store token states

Significant memory wasted in KV Cache space



Only **20% - 40%** of KV cache is utilized to store token states

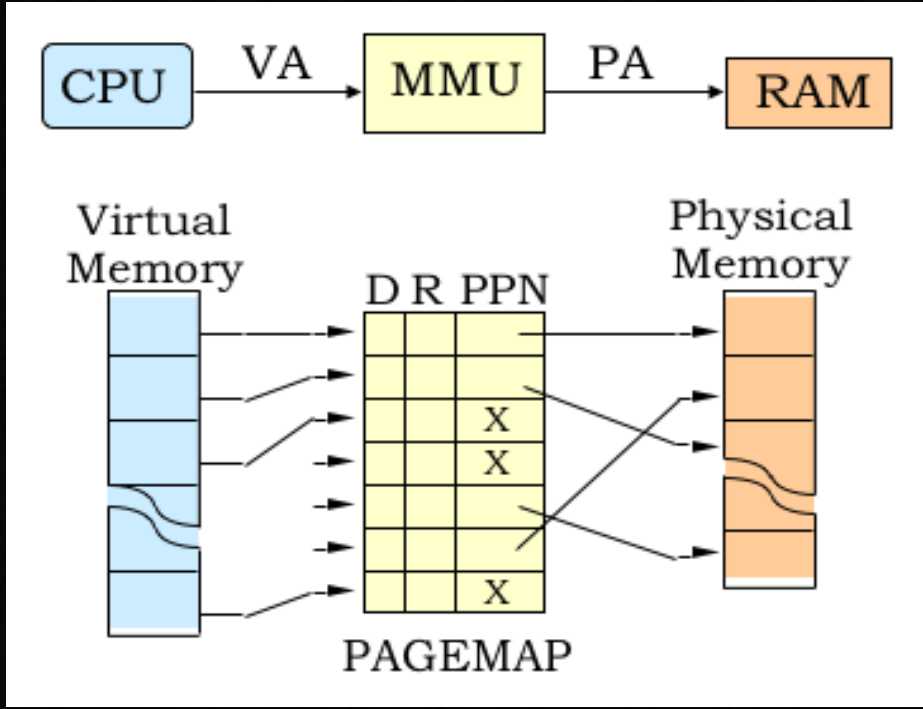
02

PagedAttention

How do Modern Operating Systems handle Fragmentation?

Virtual Memory!

- Memory that appears contiguous to CPU is mapped to non-contiguous blocks of RAM
- Memory is allocated at a page granularity
- Lazy allocation, page doesn't get allocated until the first use of memory



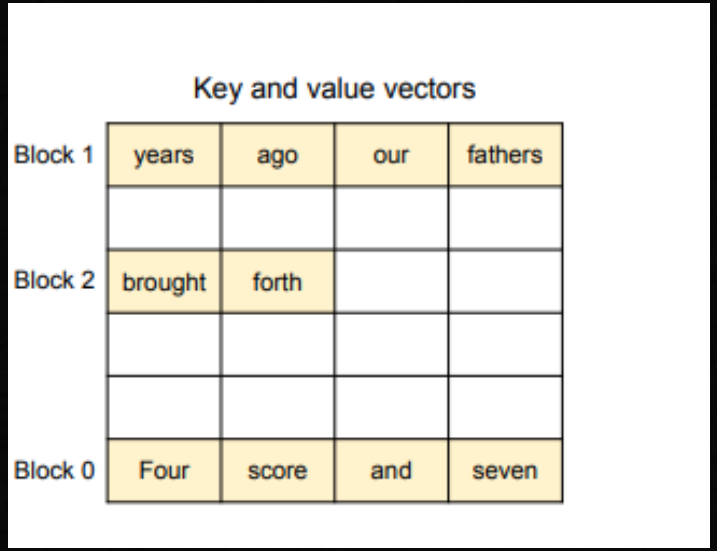
Virtual Memory in Operating Systems

Monica Spisar- Rust: Notes on memory management

KV Block Strategy

- Keep KV cache in non-contiguous, equal-sized blocks
- **No External Fragmentation**: All blocks have the same size
- **Less Internal Fragmentation**: Small block sizes
- **Less reserved space**: Lazy allocation of blocks

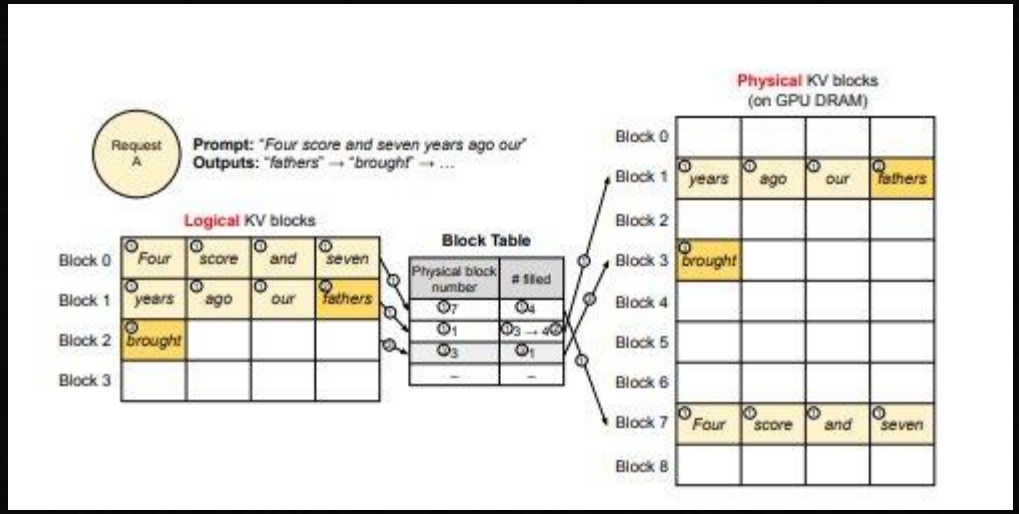
Analogy: *OS Process → Sequence of Tokens, OS page → KV block*



How KV blocks appear in memory

KV Cache Manager

- Each sequence has logical KV blocks
- Block Engine allocates contiguous chunk of GPU DRAM holding physical KV blocks
- KV Cache Manager maintains block tables, mapping logical blocks to physical blocks

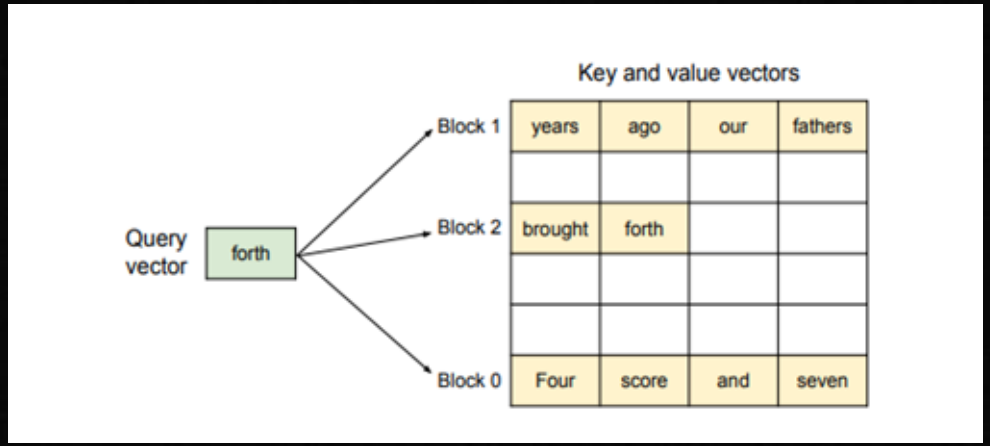


KV Cache Mapping of a Single Request

Analogy: *Virtual Address → Logical block ID, Page Table → Block Table*

PagedAttention

1. Each query attends to each of its key blocks
2. Softmax normalizes over all blocks
3. Output is a weighted sum of attention scores with the value blocks



Attention Mechanism with KV Blocks

$$A_{ij} = \frac{\exp(q_i^\top K_j / \sqrt{d})}{\sum_{t=1}^{\lceil i/B \rceil} \exp(q_i^\top K_t \mathbf{1} / \sqrt{d})}, \quad o_i = \sum_{j=1}^{\lceil i/B \rceil} V_j A_{ij}^\top,$$

Mathematical Formulation of PagedAttention

03

vLLM

What is vLLM?

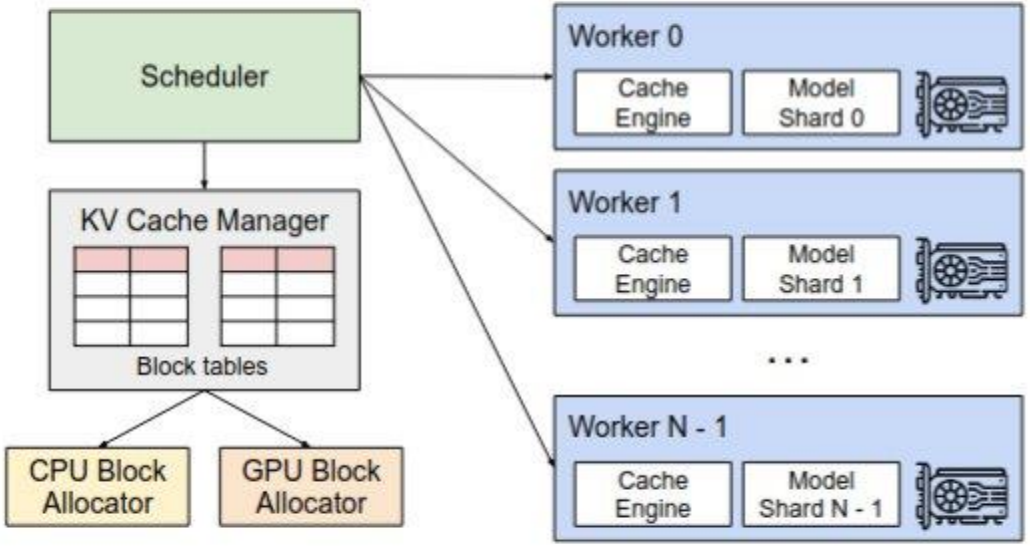


Figure 4. vLLM system overview.

Demo Example: Parallel Sampling

- Two blocks share the same input prompt and generate two output sequences
- Most blocks are shared but at some point, the sequences will differ requiring a write to a block.
- How do we efficiently handle this?
- Copy on Write!

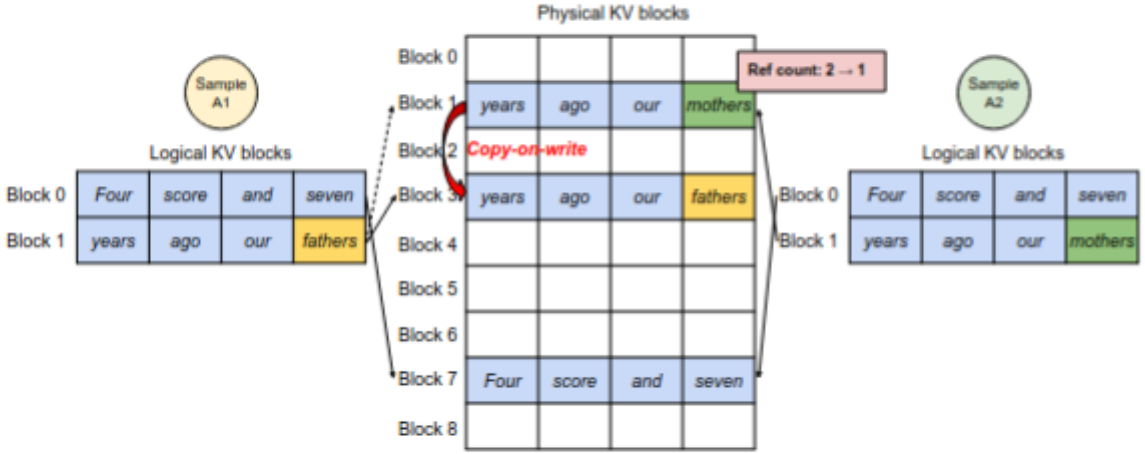


Figure 8. Parallel sampling example.

A Harder Example: Beam Search

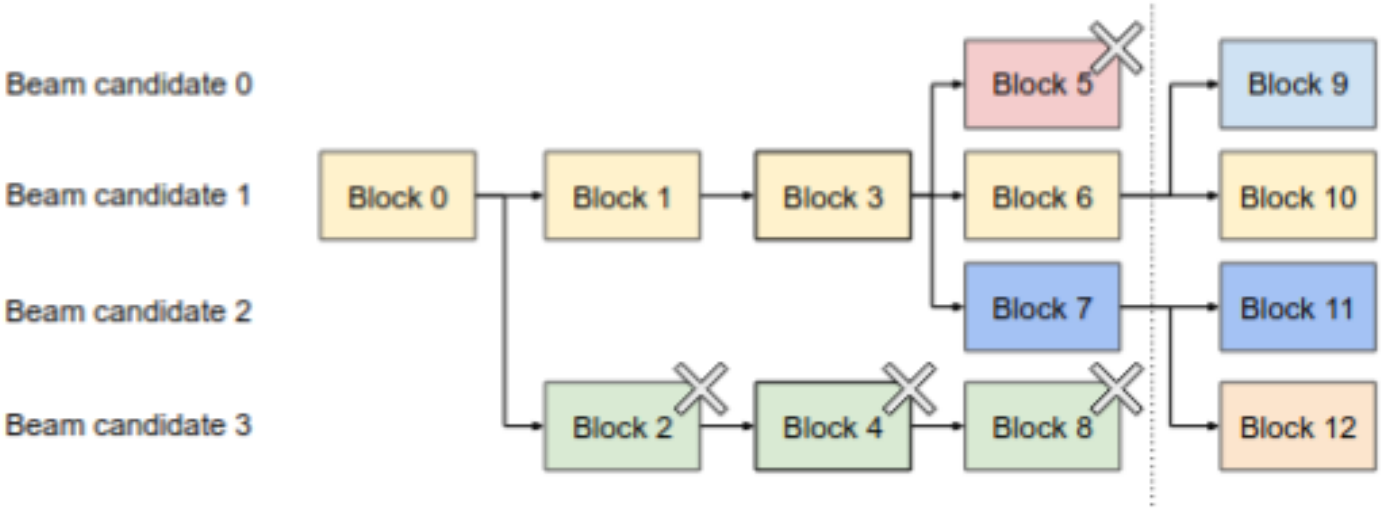


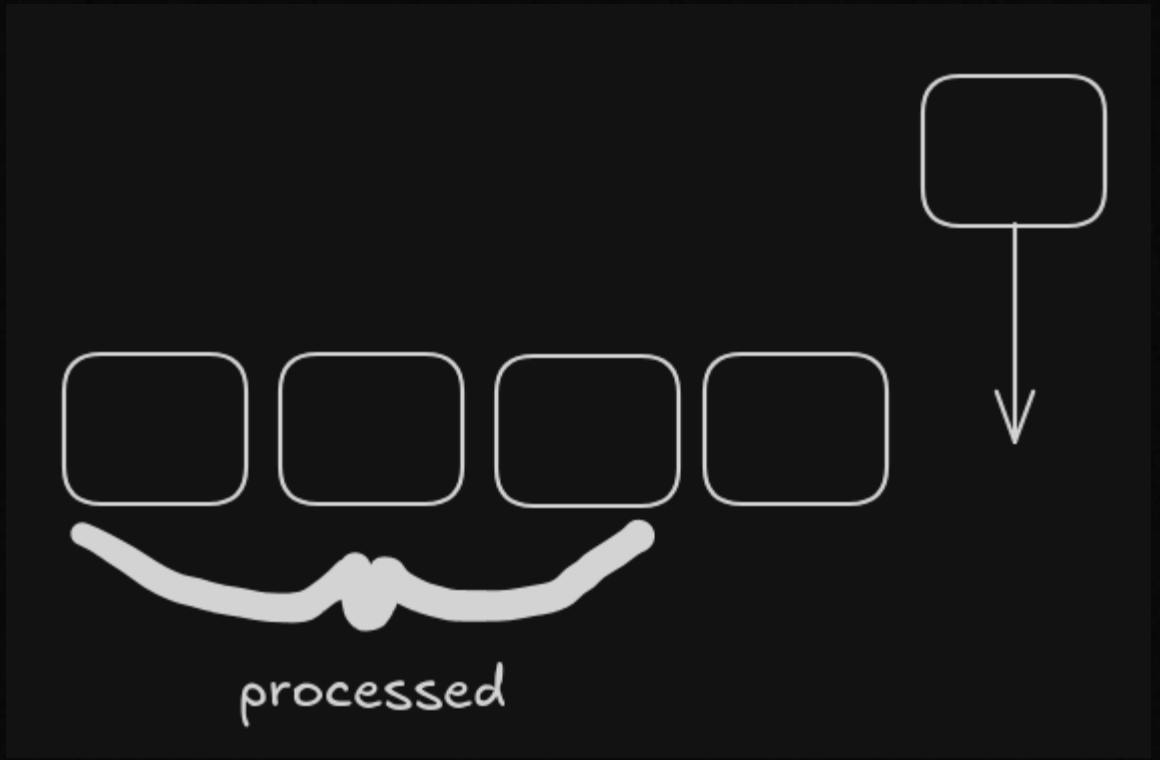
Figure 9. Beam search example.

How do we handle system prompts?

- Similar to how the OS handles shared physical pages across processes!
- vLLM reserves a set of physical pages for the system prompt's KV cache.
- This allows a LLM service provider to have a constant prompt that is used for all requests.

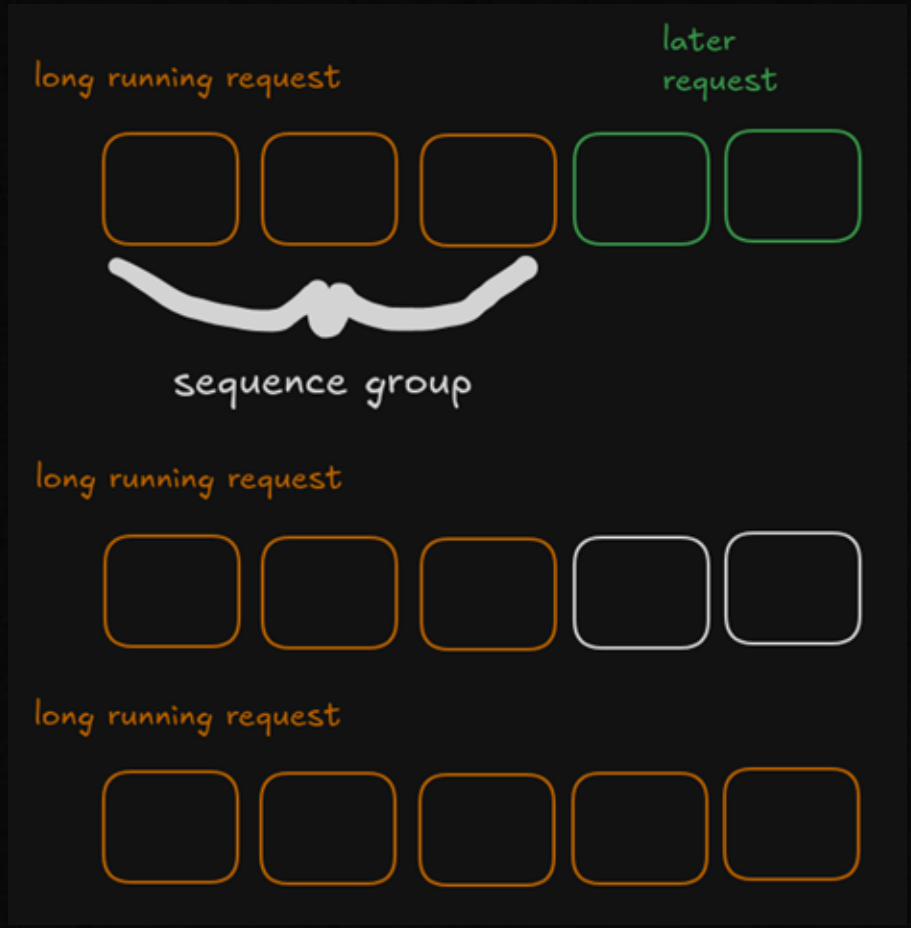
What happens if there are a lot of requests?

- When request traffic exceeds the available system capacity, vLLM has to prioritize a subset of requests.
- **First Come First Serve (FCFS) – prevents starvation**



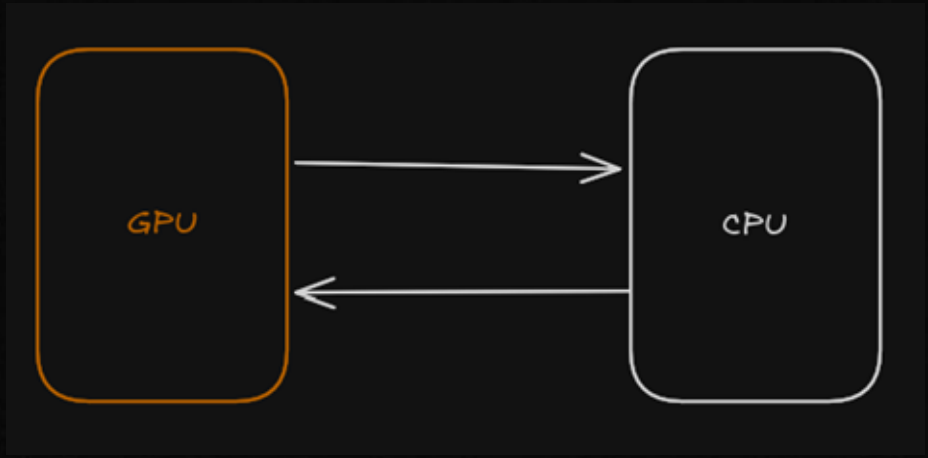
Preemption: FCFS is not all you need

- Older requests have "seniority" over the allocated memory
- These requests might be very long so we need to evict blocks and recover them
- All blocks for a request are needed so we use an all-or-nothing eviction
- Multiple sequences are scheduled together so need to be evicted together.



Block Swapping vs Recomputation

- GPUs have finite memory, so we need to swap physical blocks into the CPU
- Done via a CPU allocator
- CPU allocations never exceed GPU allocation size.
- Or we can just recompute the KV cache!
- When do you use one versus the other?



Distributed Execution

- Tensor model parallelism
- Linear layers are partitioned across GPUs as model shards.
- Intermediates are synchronized using all-reduce operations.
- KV cache is shared across GPU
- We handle this with a centralized KV cache manager and scheduler.
- Worker only stores the KV cache portion it needs.

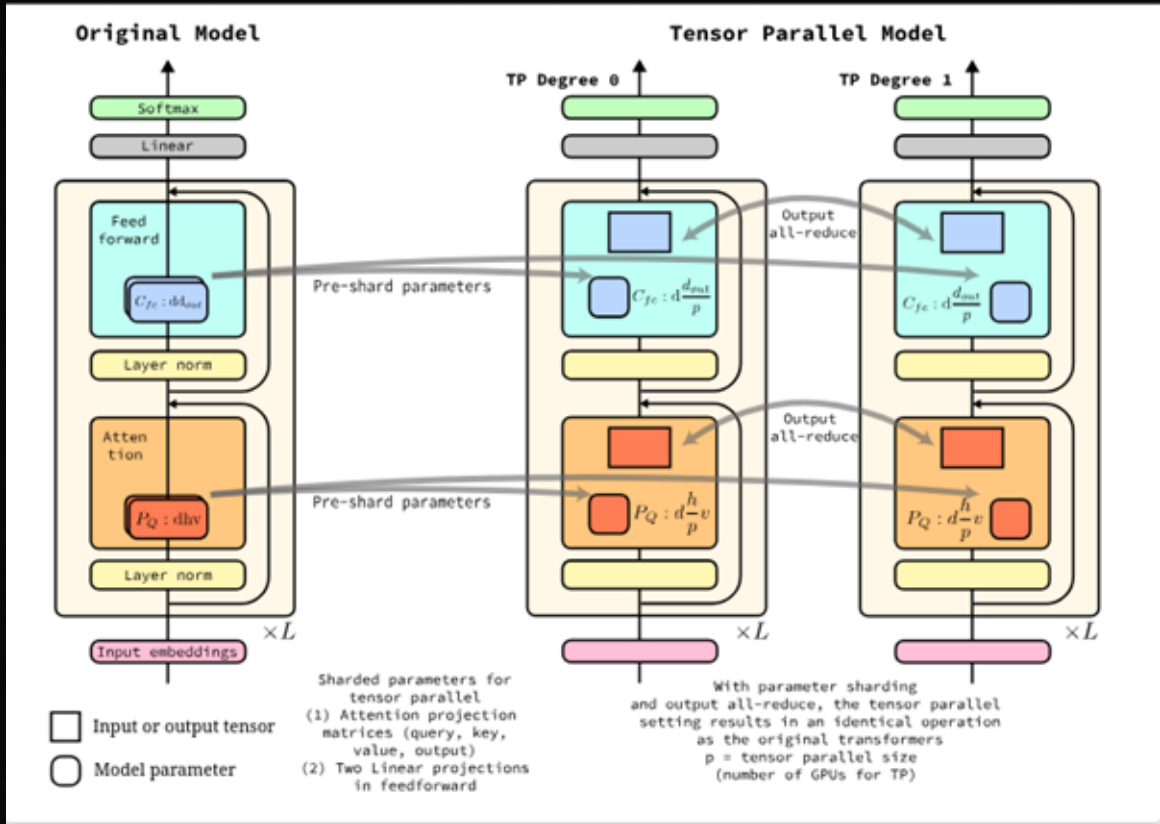


Figure 1: GPT Transformers Architecture with Tensor Parallelism

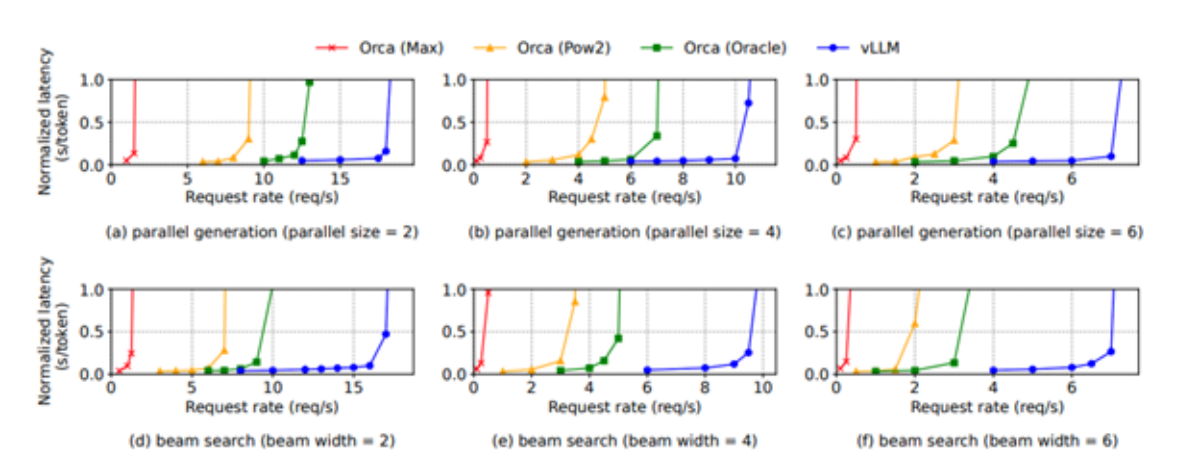
Execution Flow

- Scheduler prepares a control message with input token IDs and a block table for each request in the batch, and broadcasts it to all GPU workers
- GPU workers execute the forward pass using both the token IDs and the block table
- During attention layers, workers locate and read KV cache blocks using the block table
- All-reduce synchronizes activations across workers after each attention and FFN layer (tensor parallelism)
- Workers return the sampled tokens to the schedule

04

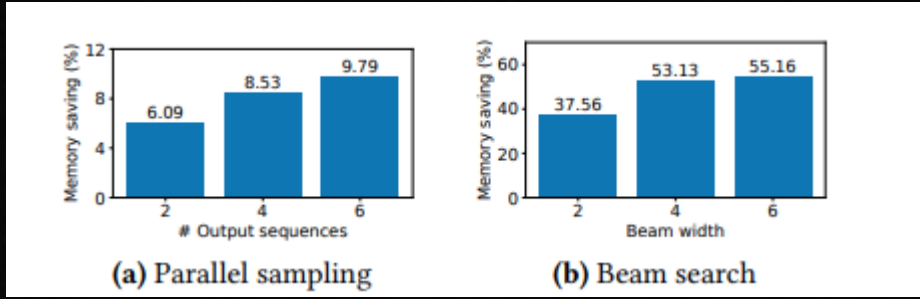
Evaluation and Insights

Throughput and Latency Improvements

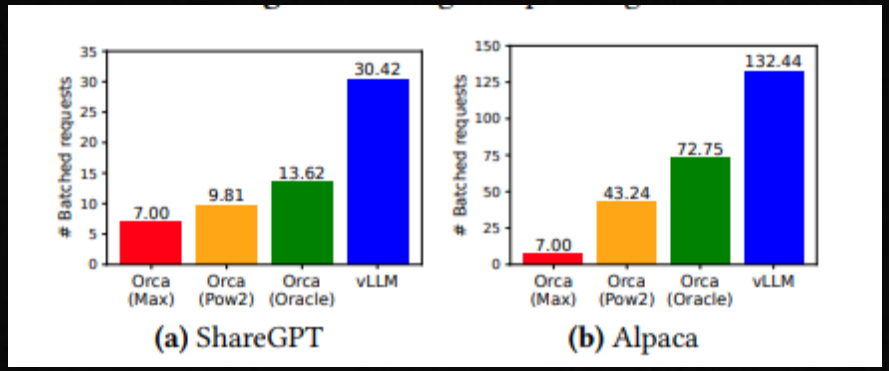


Request rate when using memory sharing for parallel generation and beam search

Memory saving from sharing KV blocks for parallel sampling and beam search

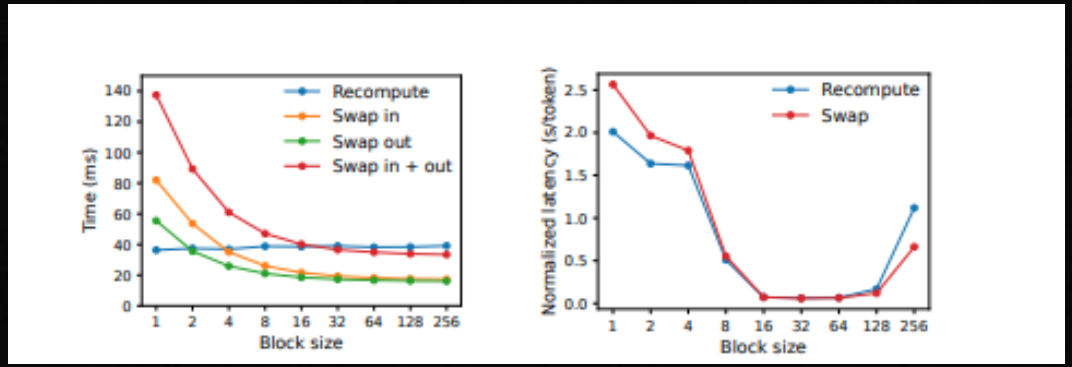


Throughput and Latency Improvements (continued) and Ablation



Average Number of Batched Requests

Overhead of Swapping and Recomputation (Left)
End-to-End Performance with this block size (Right)



Future Work

OPEN DIRECTIONS

Priority-aware scheduling

The current FCFS policy does not differentiate different types of request by priority. Some requests, like critical, real time user requests should always be served faster than background requests. Instead of FCFS, a priority-based scheduling system could be implemented, with other measures to prevent starvation.

Smarter preemption decisions

When preempted, vLLM always tries to swap before recomputing the KV cache. But as shown in ablation, it's actually faster to recompute KV cache for smaller sequences. So a naive algorithm may be suboptimal for some workloads with many small sequences. Future work could involve creating a more dynamic, cost aware preemption strategy to factor this in.

Support for Heterogeneous Memory

Right now, the KV cache manager block table only maps to the GPU DRAM. There could be potential to incorporate other types of memory like CPU RAM, for example, when GPU DRAM is getting full, instead of evicting the entire sequence, mapping the least used blocks to CPU RAM while keeping highly used blocks like system prompt blocks in the GPU.

You're serving a GPT-class model to thousands of concurrent users.

Each request generates a different number of tokens. How do you schedule the GPU?

What's the right unit of scheduling?

Orca

A Distributed Serving System for Transformer-Based Generative Models

Presented by: Aditi Ahuja and Sriram Devata

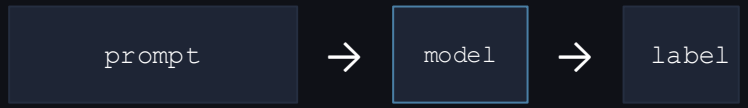
Yu, Jeong, Kim, Kim, Chun · Seoul National University / FriendliAI · OSDI 2022

Why generative inference is different

Classification / Encoding

`Input → one forward pass → output`

Fixed, predictable compute per request. Batch size = throughput knob. Requests finish in one shot — you always know when a batch is done.



One pass. Done.

Generative (autoregressive)

`Input → N forward passes → N output tokens`

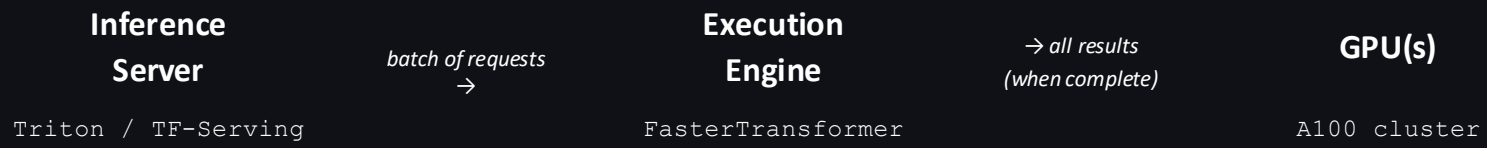
Each iteration generates exactly one token, which is fed back as input to the next. N is not known upfront — requests finish at different times.

← *generated one token at a time*

N passes × M requests = M×N separate forward passes. The model must be invoked once per token per request.

Orca · Yu et al., OSDI 2022

Existing approach: request-level scheduling



Key invariant: once the server pushes a batch to the engine, it runs until **every request in the batch** reaches its end-of-sequence token. The engine cannot add or remove requests mid-execution.

Works for classification

All requests finish in one forward pass. Batch finishes together. GPU stays busy. No straggler problem.

Breaks for generation

Requests finish at unpredictable, different times. "Wait for all" wastes GPU on done requests; new arrivals queue needlessly.

Orca · Yu et al., OSDI 2022

C1: Early-finished and late-joining requests

Two failure modes from a single rigid invariant:

1. Finished early: request holds its GPU slot until the last in-batch request finishes. GPU computes useless padding.
2. Late arrival: new request queues even when GPU has free capacity — can't join a batch already in flight.

Orca · Yu et al., OSDI 2022

C2: Batching an arbitrary set of requests

Fix C1 by scheduling at iteration granularity — run one forward pass, then pick the next batch. But that creates a new problem.

The batching constraint: attention requires all requests in a batch to produce tensors of the same shape [B, L, H]. L = sequence length. If requests have different L, you cannot form the batch.

Case 1

initiation + initiation

Both processing their prompts. Different prompt lengths → different L. Tensor shapes incompatible.

✗ **cannot batch**

Case 2

incremental + incremental

Both generating tokens. But at different positions in the sequence → different KV cache shapes. Incompatible.

✗ **cannot batch**

Case 3

initiation + incremental

One processing a prompt (all tokens at once), one generating a single new token. Completely different compute structure.

✗ **cannot batch**

Orca · Yu et al., OSDI 2022

Solutions

Two mechanisms that address C1 and C2 independently

S1: Iteration-level scheduling → solves C1

S2: Selective batching → solves C2

S1: Iteration-level scheduling - solving C1

Change the scheduling granularity from **request** to **iteration**. After every single forward pass, the scheduler decides what runs next.

1 Select requests from pool → **2** Run ONE iteration → **3** Check each for EOS → **4** Return finished requests

↺ repeat until queue empty

Fixes early-finish (C1a)

Any request that emits EOS is removed from the pool immediately after the iteration completes. It does not hold a slot waiting for other requests. Latency for short requests drops to near-optimal.

Fixes late-joining (C1b)

A new request can join the pool between any two iterations. No waiting for a batch to drain. Subject to memory constraints (KV cache slots), new requests are admitted immediately.

Orca · Yu et al., OSDI 2022

S2: Selective batching - solving C2

Since we can't batch all ops uniformly, batch only the ops that tolerate non-uniform input shapes - and handle attention separately.

Batchable ops

`Linear (Q, K, V projections)`

`Linear (FFN up/down)`

`LayerNorm`

`Residual Add`

`GeLU / activation`

These ops work on a flat 2D tensor. Doesn't matter that requests have different lengths.

Attention — run per-request

$\texttt{Attention } (QK^T / \sqrt{d}) \rightarrow \texttt{softmax} \rightarrow \times V$

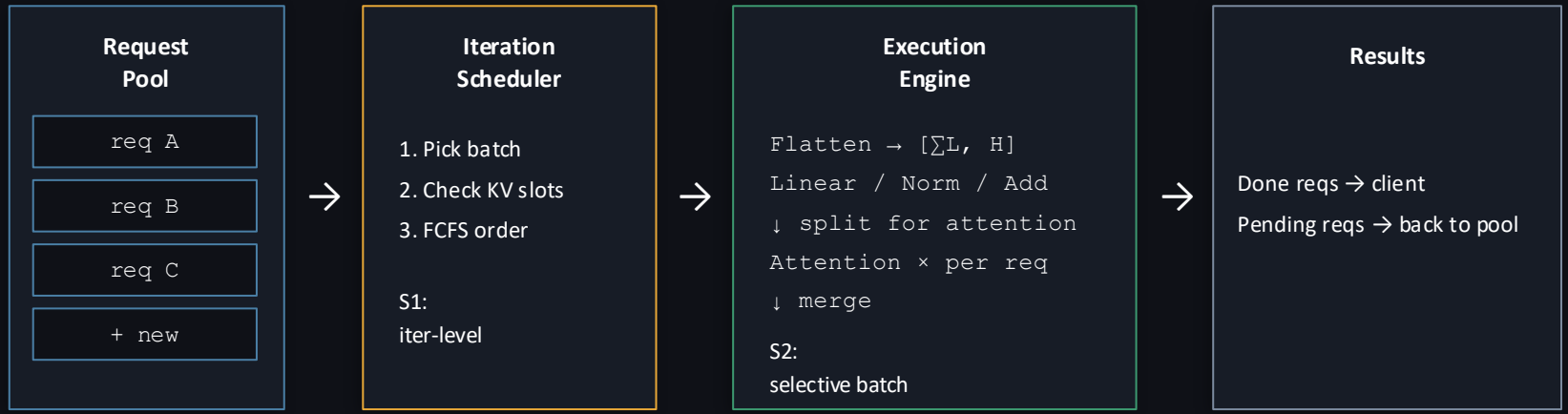
Needs [L, L] per request (sequence × sequence). Different L = can't form shared tensor. Run each request separately, then merge outputs.

Cost is small: attention is $O(L^2 \cdot d)$, but L per-request is bounded by max_tokens.

The flow: Flatten all tokens to [∑L, H] → run all non-attention ops batched → split into per-request tensors for attention → run attention separately per request → merge → continue.

Orca · Yu et al., OSDI 2022

How the two solutions compose



Net effect vs FasterTransformer baseline:

Short requests return immediately (no straggler wait). New requests join mid-flight. GPU stays fully saturated. Orca reports up to 36.9× higher throughput at equivalent latency on GPT-3 175B.

Orca · Yu et al., OSDI 2022

System Design

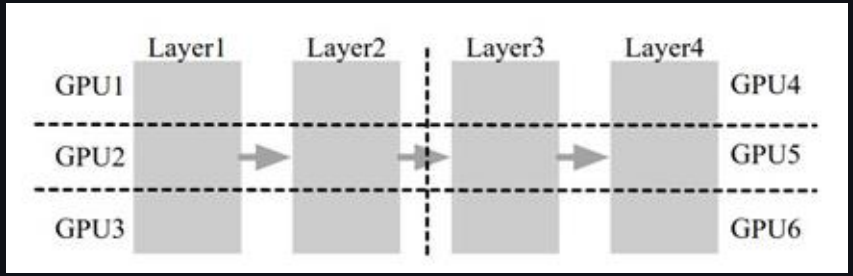
Inter and intra-layer parallelism

Inter-layer partitioning

Consecutive transformer layers split across worker groups. Each worker holds a vertical slice of the model.

Intra-layer partitioning

Within a layer, weight matrices split across GPUs in the same worker group. Each GPU handles a horizontal shard.

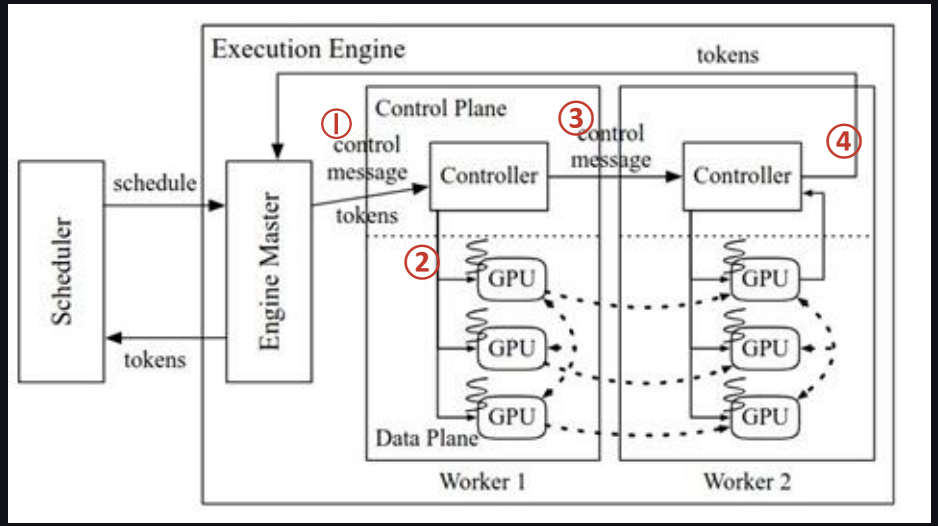


Inter-layer partitioning: Layer 1,2 and Layer 3,4

Intra-layer partitioning: Each layer is partitioned onto 3 GPUs

Orca · Yu et al., OSDI 2022

Design of Orca's execution engine



Control plane - gRPC

Data plane - NCCL

1 Engine master receives the scheduled batch from the scheduler and forwards it to Worker 1.

2 Worker 1's controller dispatches token data to its GPU-controlling threads.

3 Worker 1's controller sends a control message to Worker 2 to prepare the next stage.

4 Worker 2 waits for Worker 1's GPU kernels to finish before executing its layer partition.

Scheduling algorithm to form batches

Policy: iteration-level FCFS — but constrained by two hard limits that determine how many requests can be batched per iteration.

Diminishing returns on batch size

Larger batches improve GPU utilisation but with diminishing gains. Beyond a threshold, adding more requests increases latency without proportional throughput benefit. Scheduler caps at max_bs.

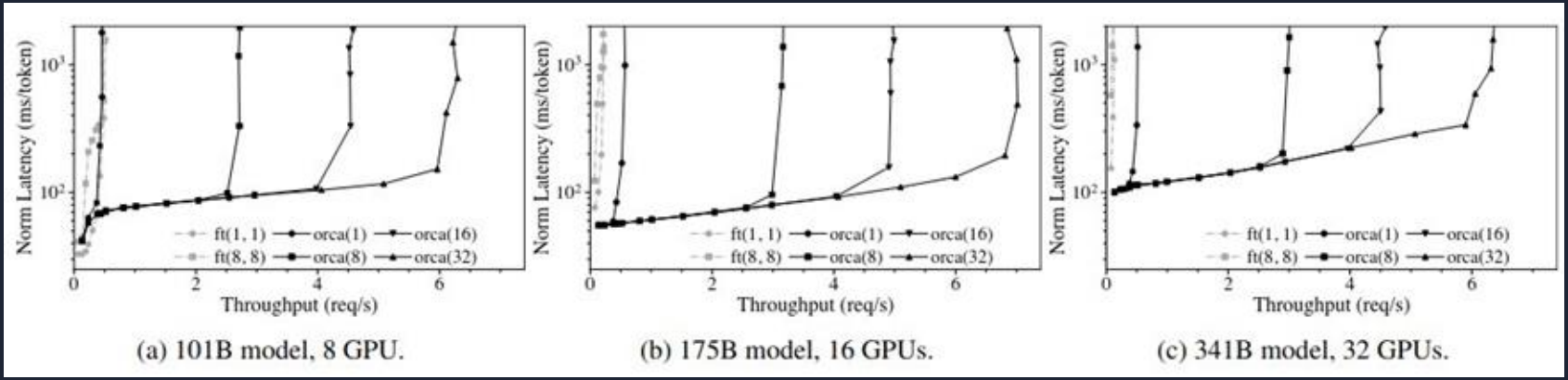
GPU memory — KV cache slots

Each active request holds KV cache for all tokens generated so far. Scheduler tracks n_slots and only admits a new request if it can guarantee memory for the full max_tokens allocation.

The ORCA execution pipeline staggers worker start times — Worker 2 begins as soon as Worker 1's iteration completes, with no batch-level synchronisation barrier. This is what enables pipelining across inter-layer partitions.

End-to-end performance

Reading the chart: x-axis = throughput (req/s), y-axis = normalised latency (ms/token, log scale). Each curve is a batch config. **Orca curves extend further right** — higher throughput at the same latency.



101B, 8 GPU: Orca extends throughput frontier well beyond FasterTransformer

175B, 16 GPU: large gap between ft and orca at equivalent latency

341B, 32 GPU: scaling holds — gains persist at largest model size

The paper in a nutshell

Problem	Request-level scheduling blocks on the slowest request in a batch and prevents dynamic admission of new requests.
C1	Early-finished requests hold their GPU slot; late arrivals queue even when capacity is free.
S1	Schedule at iteration granularity. After each forward pass, remove done requests and admit new ones.
C2	Arbitrary in-flight requests cannot be naively batched — attention requires uniform tensor shapes [B, L, H].
S2	Flatten to [∑L, H] for all non-attention ops. Split to run attention per-request. Merge for other ops.
Result	Up to 36.9× throughput improvement over FasterTransformer on GPT-3 175B at equal or better latency.

Orca · Yu et al., OSDI 2022